



ELIE ALHAJJAR, SASHA ROMANOSKY, KYLE A. KILIAN, JOE UCHILL

A Structured Approach to Identifying and Characterizing AI Vulnerabilities

For more information on this publication, visit www.rand.org/t/RR-A4983-1.

About RAND

RAND is a research organization that develops solutions to public policy challenges to help make communities throughout the world safer and more secure, healthier and more prosperous. RAND is nonprofit, nonpartisan, and committed to the public interest. To learn more about RAND, visit www.rand.org.

Research Integrity

Our research integrity is grounded in RAND's core values of quality and objectivity. Rigorous quality assurance procedures, conflict of interest screening, and transparency in funding ensure that every study is objective and nonpartisan. Learn more at www.rand.org/integrity.

RAND's publications do not necessarily reflect the opinions of its research clients and sponsors.

Published by the RAND Corporation, Santa Monica, Calif.

© 2026 RAND Corporation

RAND® is a registered trademark.

Limited Print and Electronic Distribution Rights

This publication and trademark(s) contained herein are protected by law. This representation of RAND intellectual property is provided for noncommercial use only. Unauthorized posting of this publication online is prohibited; linking directly to its webpage on rand.org is encouraged. Permission is required from RAND to reproduce, or reuse in another form, any of its research products for commercial purposes. For information on reprint and reuse permissions, visit www.rand.org/about/publishing/permissions.

About This Report

In this report, we present a vulnerability-centric framework for analyzing the security of generative artificial intelligence (AI) systems. Moving beyond lists of attacks or incidents, we systematically map where and how weaknesses arise within the architecture of large models: from training data and tokenization to transformer layers and deployment interfaces. We identify 31 classes of AI vulnerabilities, most of which differ from traditional software flaws, often emerging from probabilistic learning dynamics and optimization trade-offs rather than deterministic code errors. Using structured threat and impact metrics, we characterize risk and reveal that the greatest vulnerabilities occur during data ingestion and at user interaction boundaries. We propose practical mitigations that emphasize provenance validation, architectural safeguards, and continuous monitoring. By reframing AI security around structural weaknesses rather than adversarial techniques, we offer a foundation for integrating AI-specific vulnerabilities into global standards and for shaping future policy on trustworthy AI deployment.

Center on AI, Security, and Technology

RAND Global and Emerging Risks is a division of RAND that delivers rigorous and objective public policy research on the most consequential challenges to civilization and global security. This work was undertaken by the division's Center on AI, Security, and Technology, which aims to examine the opportunities and risks of rapid technological change, focusing on artificial intelligence, security, and biotechnology. For more information, contact cast@rand.org.

Funding

This research was independently initiated and conducted within the Center on AI, Security, and Technology using income from operations and gifts and grants from philanthropic supporters. A complete list of donors and funders is available at www.rand.org/CAST. RAND clients, donors, and grantors have no influence over research findings or recommendations.

Acknowledgments

We would like to acknowledge colleagues who reviewed this report and provided useful comments, including James Gimbi and Anna Wisakanto. In addition, we would like to thank the Common Vulnerabilities and Exposures AI and Common Weakness Enumeration AI working groups at MITRE, the Common Vulnerability Scoring System Special Interest Group, and the

organizers of the Institute for Trustworthy AI in Law and Society (TRAILS) conference (TRAILSCon) at George Washington University.

Summary

In this report, we present a vulnerability-centric framework for analyzing the security of generative artificial intelligence (AI) systems. We systematically map where and how weaknesses arise within the architecture of large models, from training data and tokenization to transformer layers and deployment interfaces. Using structured threat and impact metrics, we characterize risk and reveal that the greatest vulnerabilities occur during data ingestion and at user interaction boundaries.

Specifically, we address the following research questions:

- What are the vulnerability types in AI systems?
- What are the features of those vulnerabilities that would increase the probability of exploitation, the magnitude of impact to the AI system, and therefore the overall risk to the AI system?
- What practical mitigations could be applied to reduce the risk of malicious exploitation of these vulnerabilities?

Key Findings

Several key findings emerged from our analysis:

- First, many of the vulnerabilities in generative AI systems arise from fundamental properties of data, model architecture, and optimization objectives. Like certain architectural weaknesses in traditional computing systems, these vulnerabilities can persist across model versions and deployments and may be difficult to fully eliminate. However, unlike many conventional software vulnerabilities, AI vulnerabilities often emerge from statistical learning processes and model behavior rather than discrete implementation errors, making them more difficult to characterize, detect, and remediate.
- Second, among the AI system components examined, vulnerabilities associated with training data present the greatest overall risk. Poisoned or unverified training data can embed persistent weaknesses during model development that propagate across deployments and influence downstream behavior over time.
- Third, user-facing inference interfaces represent major points of exposure. Components such as the context window, retrieval-augmented generation pipelines, and prompt boundaries provide opportunities for adversaries to manipulate model behavior through carefully crafted inputs or injected contextual information.

These findings suggest that traditional software security approaches, particularly patch-and-fix remediation models, are insufficient for addressing many AI vulnerabilities. Because many weaknesses arise from stochastic model behavior or architectural design, mitigating risk requires a deeper

understanding of where vulnerabilities occur within the system. A vulnerability-centric analytical approach helps clarify this structure by mapping weaknesses to specific architectural components and identifying the conditions under which they can be exploited.

Applying structured threat and impact metrics further enables reproducible risk assessments by linking such factors as accessibility, exploitability, and patchability to potential confidentiality, integrity, and availability consequences, thus enabling designers to make calculated decisions that can improve the security of generative AI. The analysis also highlights the importance of optimization and reward design, because misaligned objectives or over-optimization toward proxy metrics can lead to systemic behavioral failures. At the same time, limited interpretability of internal model representations continues to hinder detection and accountability when failures occur. Finally, we identify a growing class of efficiency-based attacks that target system availability and operational stability. Such techniques as resource exhaustion or context flooding can degrade system performance and increase operational costs, demonstrating that risks to AI systems extend beyond information security to include computational and economic dimensions.

Recommendations

The following recommendations summarize practical steps that developers, security practitioners, and standards organizations can take to reduce the risk of exploitation and improve the management of AI vulnerabilities:

- **Prioritize security controls at the data layers.** Security efforts should prioritize training data, context windows, and retrieval systems, in which the highest-threat vulnerabilities are concentrated. Strong dataset governance, provenance tracking, and controls on contextual and retrieved inputs are critical.
- **Treat certain AI vulnerabilities as structural risks rather than as patchable bugs.** Because several weaknesses arise from inherent properties of machine learning systems rather than discrete implementation flaws, they often cannot be fully eliminated through conventional patching. Instead, risk must be managed through architectural safeguards, operational constraints, monitoring, and other compensating controls, which may reduce but not entirely remove the underlying vulnerability.
- **Incorporate component-level risk assessments into AI system design.** Evaluating vulnerabilities at the architectural component level helps identify where risks cluster and allows engineering teams to prioritize mitigations during system development.
- **Develop advanced exploitation detection and monitoring capabilities for AI systems.** Many exploitation pathways are difficult to detect with existing tools, making improved logging, anomaly detection, and behavioral monitoring essential for identifying adversarial activity.
- **Develop evaluation methods that reflect stochastic exploitation.** Security testing should incorporate stochastic simulations, adversarial prompting, and large-scale behavioral assessments to better capture how vulnerabilities may manifest under variable conditions.

- **Establish criteria for cataloging AI vulnerabilities.** While such initiatives as MITRE ATLAS (Adversarial Threat Landscape for Artificial-Intelligence Systems) provide valuable frameworks for documenting adversarial behaviors, clear standards are needed to determine how AI-specific weaknesses should be classified and integrated into existing vulnerability management frameworks.

Contents

About This Report.....	iii
Summary.....	v
Figures and Tables.....	x
CHAPTER 1.....	1
Introduction	1
CHAPTER 2.....	4
Research Approach	4
Methodology	4
Limitations	6
CHAPTER 3.....	8
Attacks Against AI Systems.....	8
Evasion	8
Extraction	8
Poisoning	9
Misalignment	9
Efficiency.....	9
CHAPTER 4.....	11
Enumerating AI Vulnerabilities	11
Schematic of the AI System	11
AI Vulnerabilities	12
CHAPTER 5.....	15
Characterizing the Risk of AI Vulnerabilities	15
Threat	16
Impact	18
Risk.....	20
CHAPTER 6.....	25
Mitigations for AI Vulnerabilities.....	25
Training and Fine-Tuning Data Mitigations.....	25
Tokenizer and Embedding Mitigations.....	26
Transformer Mitigations	26
Reward Function Mitigations	27
Context Window and RAG Mitigations.....	27
CHAPTER 7	29
Recommendations	29
Prioritize Security Controls at the Data Layers	29
Treat Certain AI Vulnerabilities as Structural Risks Rather Than as Patchable Bugs	29

Incorporate Component-Level Risk Assessments into AI System Design	29
Develop Advanced Exploitation Detection and Monitoring Capabilities for AI Systems	30
Develop Evaluation Methods That Reflect Stochastic Exploitation	30
Establish Criteria for Cataloging AI Vulnerabilities.....	30
CHAPTER 8	32
Conclusion	32
APPENDIX A	33
List of AI Vulnerabilities	33
Standard Listing of Vulnerabilities	33
List of Vulnerabilities in JSON Format	42
APPENDIX B	55
Explanation of Threat and Impact Scores.....	55
Abbreviations	60
References	61
About the Authors	68

Figures and Tables

Figures

Figure 1.1. Research Contributions of This Report	2
Figure 2.1. Research Approach.....	4
Figure 4.1. AI System Schematic.....	11
Figure 4.2. Annotated AI System Schematic.....	13
Figure 5.1. Detailed Threat and Impact Results, by AI System Component.....	21
Figure 5.2. Aggregate Threat and Impact Results, by AI System Component.....	23

Tables

Table 3.1. Attack Categories.....	10
Table 5.1. Threat Metrics	16
Table 5.2. Threat Rubric.....	17
Table 5.3. Impact Metrics	18
Table 5.4. Impact Rubric.....	20
Table 6.1. Mitigations for AI Vulnerabilities	28
Table B.1. Threat Assessment Explanation, by AI System Component	55
Table B.2. Impact Severity Assessment Explanation.....	57

Introduction

The pace of development in artificial intelligence (AI) capabilities, coupled with the rapid adoption of these systems across consumer, enterprise, and government applications, has been unprecedented. Large language models (LLMs) and other generative AI systems are now widely integrated into search engines, productivity software, coding tools, customer support platforms, and scientific research workflows. This rapid diffusion of AI technologies has been accompanied by significant optimism regarding their potential economic and societal benefits. However, the speed of deployment has also outpaced our collective understanding of the risks that these systems introduce and the mechanisms by which they may fail or be exploited (Bommasani et al., 2022; Brundage et al., 2024).

In response, a growing ecosystem of industry, academic, and civil society initiatives has emerged to document and analyze the risks associated with AI systems. A growing body of research literature describes adversarial attacks, prompt injection techniques, data poisoning strategies, and other mechanisms by which AI systems can be manipulated (Alhajjar, Maxwell, and Bastian, 2021; Ghosh et al., 2025; Vassilev et al., 2025).

Much of the existing literature and data collection focuses on attacks, incidents, or misuse cases rather than on the underlying vulnerabilities that make such attacks possible. That is, many studies describe how AI systems are attacked, but fewer systematically identify the specific structural weaknesses within AI systems that enable those attacks to succeed. Without a clear conceptualization of these vulnerabilities, it becomes difficult to reason systematically about risk, compare mitigation strategies, or integrate AI security issues into existing vulnerability management frameworks.

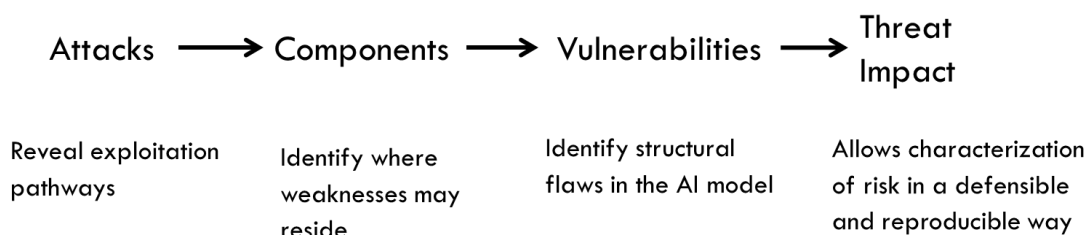
Part of the difficulty lies in the fact that vulnerabilities in AI systems challenge conventional notions of software vulnerabilities. In traditional software security, vulnerabilities are typically identifiable as specific flaws in program logic or implementation, often localized to particular lines of code, and their behavior is largely deterministic (Dowd, McDonald, and Schuh, 2006). These weaknesses can often be addressed through code revisions, configuration changes, or software patches. Over decades, the software security community has developed robust mechanisms for cataloging and managing such vulnerabilities, including the Common Vulnerabilities and Exposures (CVE) system and the Common Weakness Enumeration (CWE) taxonomy (CVE Program, undated; CWE, undated). Risk associated with these vulnerabilities is often quantified using standardized scoring systems, such as the Common Vulnerability Scoring System (CVSS) and the Exploit Prediction Scoring System (EPSS), which characterize both the likelihood of exploitation and the severity of potential impacts (Forum of Incident Response and Security Teams, undated-a; Forum of Incident Response and Security Teams, undated-b).

AI vulnerabilities differ from traditional software vulnerabilities in several important respects. First, many AI failures arise from statistical properties of trained models rather than deterministic

code defects. Weaknesses may emerge from training data composition, representational structures within the model, optimization objectives, or deployment architectures. Second, AI vulnerabilities often manifest probabilistically rather than deterministically; the same input may produce different outputs depending on model stochasticity or contextual information. Third, some weaknesses in AI systems may not be fully patchable in the conventional sense because they arise from fundamental trade-offs in statistical learning and model architecture, such as generalization, representation capacity, or alignment between training objectives and deployment contexts. As a result, AI security challenges require conceptual tools that differ from those traditionally used in software vulnerability management (Open Worldwide Application Security Project [OWASP] Foundation, undated-a).

In this report, we seek to address this gap by developing a systematic framework for identifying and analyzing vulnerabilities in generative AI systems (Figure 1.1). For the purposes of this research, we define a *vulnerability* as a flaw or weakness in an AI system component that could be exploited to compromise the confidentiality, integrity, or availability of the inputs to or outputs from that system. This definition extends traditional vulnerability concepts to account for the fact that, in AI systems, harms may arise not only from unauthorized access to system components but also from manipulation or degradation of model behavior (Aslan et al., 2023).

Figure 1.1. Research Contributions of This Report



A central contribution of this work is the adoption of a vulnerability-centric approach to AI security analysis. Rather than focusing primarily on specific attack techniques, we seek to identify the underlying weaknesses within AI systems that enable multiple classes of attacks to succeed. By examining where these weaknesses arise within the architecture of generative AI systems and how they influence the likelihood and impact of exploitation, we aim to provide a more structured foundation for understanding and mitigating AI security risks.

Specifically, we address the following research questions in this report:

- What are the vulnerability types in AI systems?
- What are the features of those vulnerabilities that would increase the probability of exploitation, the magnitude of impact to the AI system, and therefore the overall risk to the AI system?
- What practical mitigations could be applied to reduce the risk of malicious exploitation of these vulnerabilities?

To answer these questions, we developed a structured decomposition of generative AI systems, identifying the major components involved in both their training and deployment. This schematic representation provides a principled basis for locating vulnerabilities within the AI stack. Building on this structure, we clustered attacks against AI systems into five broad categories and analyzed how these attacks exploit underlying system weaknesses. Across these components, we enumerate more than 30 distinct classes of vulnerabilities, providing a characterization of where and why generative AI systems are susceptible to exploitation.

To assess the practical significance of these vulnerabilities, we introduce a set of structured metrics that capture two key dimensions of risk: the likelihood that a vulnerability can be successfully exploited and the severity of the impact if exploitation occurs. By combining these dimensions, we developed a reproducible framework for evaluating the overall risk associated with different classes of AI vulnerabilities. Finally, we propose criteria that could inform how AI-specific vulnerabilities might be incorporated into existing vulnerability standards, such as CVE and CWE, thereby contributing to the development of more-systematic mechanisms for cataloging and managing AI security weaknesses.

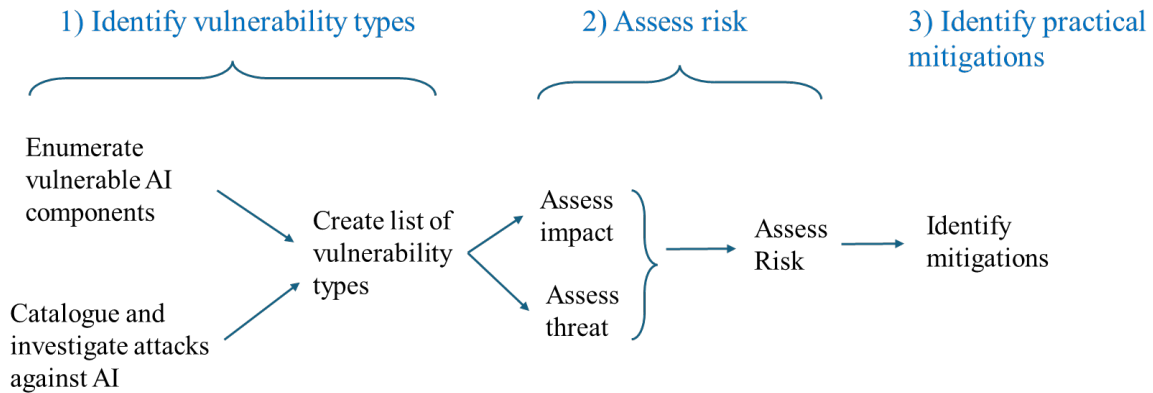
Together, these contributions aim to move the discussion of AI security beyond lists of attacks or incidents toward a more rigorous understanding of the structural vulnerabilities embedded within modern AI systems. This report is organized as follows: In Chapter 2, we describe the research approach used to derive vulnerabilities and analyze their associated risks. Chapter 3 presents a taxonomy of attacks against AI systems. Chapter 4 enumerates vulnerabilities across the major components of generative AI systems. Chapter 5 introduces a framework for characterizing risk by evaluating both the likelihood of exploitation and the severity of potential impacts. Chapter 6 discusses mitigation strategies applicable to different classes of vulnerabilities. We finish the report with a set of recommendations in Chapter 7 and a brief conclusion in Chapter 8.

Research Approach

Methodology

We outline the research approach in Figure 2.1 and explain it below.

Figure 2.1. Research Approach



To identify the set of possible vulnerabilities in AI systems, we applied two approaches. First, we examined prominent literature of AI systems, model diagrams, and other reference material and constructed a schematic that identifies each component of an AI system, from the training of a base model to the deployment of a functional large model (depicted in Figure 4.1 in Chapter 4).

Then, for each component, we documented the flaws or weaknesses that could compromise the operation of the AI system or the information generated from it. Next, we collected examples of both real-world and theoretical attacks against AI systems by monitoring multiple cyber and technology mailing lists daily from August 2025 through March 2026 and by reviewing blogs, news stories, and research papers. For each attack, we coded the name of the attack (if named), the publication date, the source, the type of attack, the weakness(es) exploited in the AI system (where possible), and whether it was a real-world or a theoretical attack. These two steps combined to produce our final list of vulnerability types.

We defined a set of scoping decisions for this research:

- Our focus is on *generative* AI, which we define as large models that generate text, image, video, speech, and other elements. We acknowledge that the focus of most literature is on LLMs, although many of the vulnerabilities identified in our analysis stem from properties of data-

driven learning, representation, optimization, and deployment that are also present in multimodal generative models.

- In our analysis of AI risk, we did not explicitly consider harms resulting from AI bias, such as racially, demographically, or politically biased outputs.¹
- We did not examine attacks *using* AI tools but rather just vulnerabilities *in* AI systems.

In our analysis of threats and impacts, we considered direct impacts *to the AI model* and did not include indirect impacts to external systems. Consequently, we did not examine vulnerabilities in computing, networking, or supply chain components.

We need to highlight two important notes. First, this list represents vulnerability *types*, or *classes* of vulnerabilities, not *specific* vulnerabilities in *specific* AI systems. Just as the CWE schema defines *classes* of regular software vulnerabilities (with CVEs being specific instantiations of a vulnerability class in a specific product version), the AI vulnerability types we discuss in this report are types or classes.

With a working list of vulnerability types, we next sought to characterize the features *of the vulnerability* that would affect the probability of exploitation (threat) and the severity of impact *to the AI system* if a vulnerability of this type were to be exploited. This approach was heavily influenced by the efforts of the CVSS and its approach to enumerating features of the vulnerability that affect these risk metrics (Chapter 5).

Through our literature review and vulnerability analysis, we identified and captured mitigations that could be applied to each AI vulnerability. We first investigated and identified vulnerabilities across each component of AI development and deployment, beginning with training, then fine-tuning, followed by inference and deployment. We also integrated mitigations from the Secure AI Framework (SAIF), segmenting vulnerabilities by life-cycle stage and categories, such as governance, assurance, data, and structure (or infrastructure) (Google, undated). This analysis reveals how some mitigations can be applied over multiple phases, while other mitigations address only a single phase or are relevant to only a single component.

This research differs in both scope and methodology from existing efforts that catalog AI security concerns. AI incident databases, for example, compile news reports and documented cases of AI systems causing harm. Although valuable for tracking real-world impacts, such efforts focus on outcomes rather than the underlying structural weaknesses that make those outcomes possible. Our objective was not to aggregate incidents but to identify and characterize the architectural vulnerabilities that give rise to exploitable risk.

Other initiatives, such as the OWASP Top 10 for LLMs, provide practitioner-oriented lists of common attack types (OWASP Foundation, undated-b). These lists are useful for awareness and defensive guidance but are not structured around distinct vulnerability classes. Many listed items overlap conceptually or map to multiple underlying weaknesses. Moreover, such efforts tend to focus specifically on LLMs, whereas our framework considers generative AI systems more broadly, including multimodal models and systems augmented with retrieval and memory components.

¹ We did not attempt to comprehensively address the broader AI safety topics of loss of control, deceptive alignment, power-seeking behavior, or other forms of advanced agentic misalignment. However, we do consider component-level vulnerabilities associated with objective misspecification and related mechanisms where they contribute directly to confidentiality, integrity, or availability failures.

Importantly, our approach emphasizes a systematic derivation of vulnerabilities from architectural decomposition rather than a curated enumeration of prominent attack patterns.

Similarly, MITRE ATLAS (Adversarial Threat Landscape for Artificial-Intelligence Systems) offers a detailed taxonomy of adversarial machine learning techniques, organized around tactics, techniques, and procedures across an attack life cycle (MITRE ATLAS, undated). This work is valuable for threat modeling and red teaming, but it is attack-centric. In contrast, our analysis is vulnerability-centric: Rather than cataloging how adversaries operate, we sought to identify the structural weaknesses within generative AI systems that enable exploitation across multiple attack classes.

In the same context, the National Institute of Standards and Technology (NIST) AI Risk Management Framework provides a comprehensive governance framework for identifying, managing, and communicating risks associated with AI systems (NIST, undated). It emphasizes organizational processes, such as risk governance, measurement, monitoring, and documentation, across the AI life cycle. While this approach is valuable for establishing institutional practices and accountability, it operates at a high level of abstraction and does not attempt to enumerate specific technical vulnerabilities within AI system architectures. In contrast, our analysis focuses on the structural components of generative AI systems and systematically derives vulnerability classes from those components. Rather than providing general guidance for risk management, our framework was designed to identify the specific technical weaknesses that may give rise to the risks that organizations are expected to manage under broader governance frameworks.

Limitations

We do not claim to provide an exhaustive catalog of all possible vulnerabilities across all AI systems but rather vulnerabilities that relate to observed attacks at the time of writing. Generative AI architectures continue to evolve rapidly, and new modalities, system integrations, and deployment configurations may introduce novel weaknesses that are not captured in this report. Our enumeration should therefore be understood as a structured baseline rather than as a definitive taxonomy. The framework is designed to be expandable, allowing additional vulnerability classes to be incorporated as systems and threat models mature.

Enumeration of vulnerabilities is inherently imperfect. Vulnerabilities do not exist as neatly separable natural categories; rather, they often overlap conceptually, share underlying mechanisms, or differ in only degree or context. In some cases, what appears to be a distinct vulnerability may instead represent a variant or manifestation of a more general weakness. Conversely, seemingly similar vulnerabilities may have materially different implications depending on system architecture or deployment context. In developing this classification, we sought to balance completeness with usability, avoiding both excessive granularity that obscures structural patterns and over-aggregation that masks meaningful distinctions.

A related limitation concerns the architectural decomposition of AI systems into components. The boundaries between components, such as tokenizer, embedding, and transformer layers, are analytically useful but not always cleanly separable in practice. Modern AI systems increasingly blur distinctions between training and inference, model and tool use, core architecture and orchestration

layer. Our component framework reflects a conceptual decomposition intended to facilitate vulnerability mapping, not a rigid representation of every system's implementation. Alternative decompositions may be equally defensible depending on analytical objectives.

Additionally, the analysis is limited to vulnerabilities that arise from the architecture and operation of AI systems themselves. It does not address risks originating from external factors, such as supporting infrastructure, supply chains, hardware platforms, organizational processes, or broader governance failures. Consequently, the risk landscape presented here reflects only the subset of risks that are intrinsic to AI system components and their interactions.

Despite these limitations, the vulnerability-centric approach presented in this report produces a structured framework for analyzing AI vulnerabilities by linking architectural components, vulnerability classes, risk characteristics, and mitigation strategies within a common analytical structure.

Attacks Against AI Systems

Generative AI systems are exposed to a broad and evolving landscape of exploitation techniques. These techniques vary in timing, mechanism, required capability, and intended effect. Some involve direct adversarial manipulation of inputs or training data; others exploit inherent statistical or behavioral properties of the model without modifying its parameters. This chapter introduces a high-level categorization of attacks against AI systems. We use these categories as a diagnostic tool to illustrate how diverse attack techniques map onto a shared set of underlying vulnerabilities. We identify five broad categories of exploitation: evasion, extraction, poisoning, misalignment, and efficiency. Together, these categories span both training time and inference time threats, as well as manipulation of system boundaries and internal representations.

Evasion

Evasion attacks manipulate inputs at inference time to cause the model to produce unintended or unsafe outputs without altering its underlying weights (Hackett et al., 2025). In generative AI systems, these include techniques such as (direct or indirect) prompt injection, adversarial context manipulation, encoding tricks that bypass filters, and crafted token sequences that distort semantic interpretation. Evasion attacks exploit weaknesses in how models process and prioritize input information. These attacks do not require access to training data or model internals, making them particularly relevant in widely deployed application programming interface (API)–based systems.

Extraction

Extraction attacks aim to recover information, capabilities, or functionality from a model. These include membership inference, reconstruction of memorized training examples, model extraction, capability distillation, and attempts to reverse engineer model parameters or proprietary knowledge (Das, Amini, and Wu, 2025). Some extraction attacks exploit the model’s tendency to encode and occasionally reproduce artifacts from its training corpus, whereas others seek to replicate the model’s learned behavior or capabilities through repeated interaction. In large generative systems trained on heterogeneous data, memorization and overconfidence may enable partial disclosure of sensitive or proprietary information. Extraction can occur even in the absence of direct access to internal weights, relying solely on repeated inference queries.²

² Among extraction attacks, distillation attacks have garnered a lot of attention lately. More specifically, adversaries can train models using U.S. frontier model outputs in a process known as *distillation* to replicate capabilities at a potentially reduced cost (Mansourian et al., 2025).

Poisoning

Poisoning attacks target the training or fine-tuning pipeline by introducing manipulated data designed to alter model behavior. They may include backdoor triggers, label manipulation, or systematic insertion of biased or malicious examples (Fendley et al., 2025). Poisoning exploits weaknesses in data validation and governance processes. Because large-scale training pipelines often rely on automated data collection and aggregation, subtle manipulations can persist undetected and become encoded in model parameters. Unlike evasion, poisoning operates upstream and can produce persistent behavioral shifts that manifest long after deployment.

Misalignment

Not all exploitation requires adversarial modification of inputs or training data. Generative models exhibit inherent behavioral tendencies, such as hallucination, overconfidence, or sycophancy, that may be leveraged or amplified in deployment (Zhou et al., 2023). Here, the system’s statistical or optimization-driven properties produce outputs that may be unreliable, misleading, or misaligned with stakeholder intent. Although these weaknesses may arise without malicious intervention, they can be operationalized by adversaries or result in systemic harm in high-stakes settings.³ This category highlights an important distinction: Some risks are induced externally, whereas others are intrinsic to the model’s design and training dynamics.

Efficiency

Efficiency-based attacks aim to degrade system performance, exhaust computational resources, or induce denial-of-service conditions (Brachemi Meftah et al., 2026). Examples include token flooding, excessive context expansion, and repeated high-cost queries intended to increase inference time or resource consumption. These attacks exploit architectural constraints, such as context window size, memory allocation, and inference scaling properties. Although they do not necessarily alter model outputs directly, they affect system availability and operational stability.

Table 3.1 summarizes the five broad categories of exploitation attacks.

³ *Reward hacking behavior* occurs when a model learns to optimize the proxy objective represented by the reward function rather than the underlying task objective. An adversary may exploit this tendency by crafting prompts that induce behavior favored by the reward model but that are inconsistent with the developer’s intent.

Table 3.1. Attack Categories

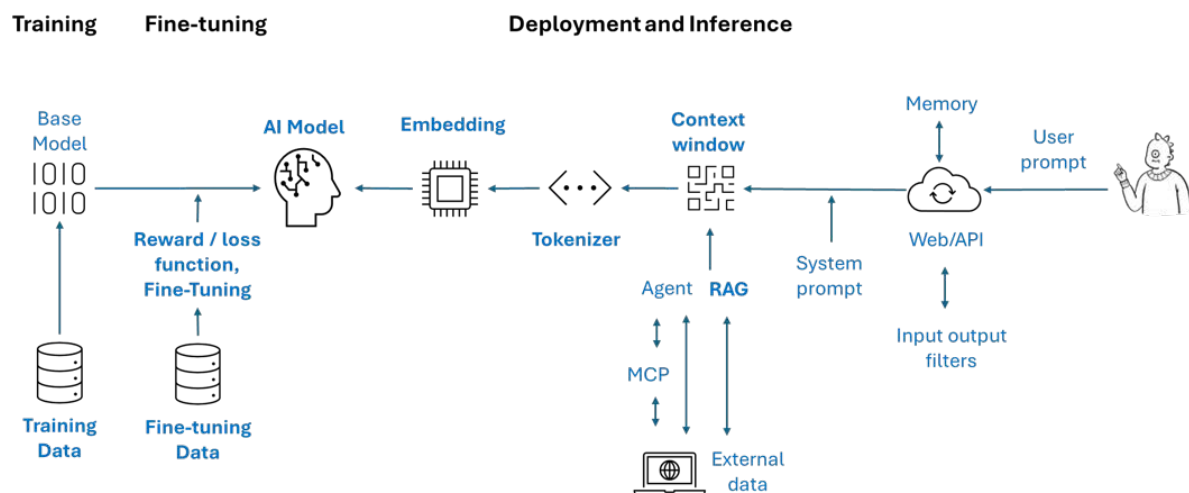
Attack Type	Description	Examples
Evasion	Manipulating inputs to produce incorrect or harmful outputs or solicit restricted AI model information without altering the underlying model	• Direct prompt injection • Indirect prompt injection • Jailbreaking
Extraction	Gaining unauthorized access to training data or model weights through strategic interactions with the model	• Prompt leaking • Model extraction • Membership inference
Poisoning	Corrupting data during ingestion by the model to affect its intended behavior	• Data poisoning • Model poisoning • Backdoor attacks
Misalignment	Exploiting, without necessarily causing, incorrect or deceptive model outputs	• Hallucination exploitation • Bias amplification • Lying, scheming, or deception
Efficiency	Degrading system performance, inflating costs, or causing denial-of-service conditions without needing to access internal model components	• Context window overflow • Energy latency attacks

Enumerating AI Vulnerabilities

Schematic of the AI System

To systematically enumerate vulnerabilities, we first decompose the AI system into its constituent components (Figure 4.1). This decomposition is not arbitrary; it reflects both the architectural and operational components of an AI system, as well as the life cycle of model development and deployment, from data ingestion through inference and system integration. We distinguish between components involved in model development and those involved in deployment and runtime interaction. This component-based decomposition enables a structured analysis: Rather than enumerating isolated failures, we identify the location within the AI stack where a weakness resides. This allows us to distinguish between transient runtime issues and vulnerabilities rooted in training-time design choices.

Figure 4.1. AI System Schematic



NOTE: There is also a reward-loss function that occurs at training (pre-training), although we omit it for clarity. MCP = Model Context Protocol; RAG = retrieval-augmented generation.

Training and Fine-Tuning Components

These components include the training data, fine-tuning data, tokenizer, embedding layer, transformer weights, reward-loss function, and the fine-tuning process itself.⁴ Together, these components determine what the model learns, how it represents information, and which behaviors are reinforced. Vulnerabilities at this stage are often structural and persistent because they become embedded in the model's parameters.

Deployment and Inference Components

These components include the context window, memory mechanisms, API layer, tokenizer, embedding layer, RAG systems, and input and output filters. They govern how the model interacts with users, tools, and external data sources. Vulnerabilities here are often interface-based and may be more amenable to mitigation through architectural controls or monitoring.

Although the schematic is intended to depict a complete generative AI system, we do not enumerate vulnerabilities associated with system memory, web or API infrastructure, and input and output filtering mechanisms in this analysis.⁵ Weaknesses in these components largely reflect conventional software security flaws that are already well characterized within existing cybersecurity standards and practices. Our focus instead is on vulnerabilities that are intrinsic to the statistical, architectural, and optimization properties of generative AI systems themselves.

In addition, we do not address vulnerabilities arising from agentic AI architectures or related orchestration frameworks (including, for example, MCP integrations). These systems introduce qualitatively different risk considerations, particularly around autonomy, tool use, and multistep decisionmaking, that extend beyond the scope of this analysis. A systematic treatment of vulnerabilities specific to agentic and tool-augmented AI systems remains an important direction for future research and will be the subject of an upcoming project.

AI Vulnerabilities

With the system decomposed into components, we can identify classes of vulnerabilities as structural weaknesses within those components rather than as specific instantiations of vulnerabilities in specific AI models.⁶ Importantly, vulnerabilities are not synonymous with attacks. An *attack* is a method of exploitation; a *vulnerability* is the underlying weakness that makes exploitation possible. Multiple attack types may map to the same vulnerability, and a single vulnerability may manifest across distinct exploitation modes. We group vulnerabilities according to the component in which

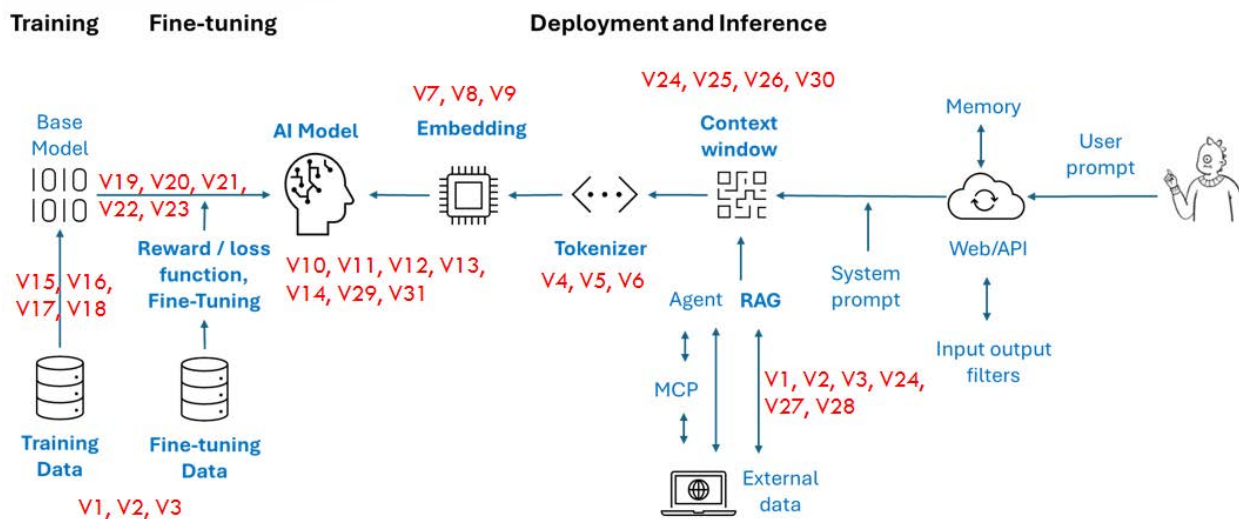
⁴ The *tokenizer* is responsible for splitting an input text into tokens. The *embedding layer* takes the list of tokens produced by the tokenizer and produces an embedding for each token. The *transformer weights* are the core component of modern LLMs; they take the embeddings and mix them together to provide context.

⁵ *Input and output filters* refers here to traditional software tools used to filter information flows. However, they can sometimes be realized as smaller AI models (e.g., LLMs as judges). In this case, the vulnerabilities in such components follow our analysis of the general AI system.

⁶ This is the same distinction between a CWE and a CVE. A CWE entry describes a class or type of vulnerability, whereas a CVE ID identifies a specific vulnerability (of a given CWE type) in a specific software package.

they arise (see Figure 4.2). By shifting the focus from attacks to vulnerabilities, we establish the foundation for the subsequent risk characterization framework. A detailed description of the identified vulnerabilities with a clear mapping to AI system components appears in Appendix A.

Figure 4.2. Annotated AI System Schematic



Vulnerabilities in Training and Fine-Tuning Data

At the training and fine-tuning stages, vulnerabilities emerge from weaknesses in data governance and validation. These vulnerabilities include unverified provenance, the presence of manipulated or poisoned data, and a lack of quality control (Zhu et al., 2024). Because large models ingest vast and heterogeneous corpora, insufficient validation can introduce persistent structural weaknesses, including backdoors, memorization of sensitive information, or skewed behavioral priors. Data-level vulnerabilities are particularly consequential because they become encoded in the model's weights and are difficult to remove post hoc without retraining.

Tokenizer and Embedding Vulnerabilities

Within the tokenizer and embedding layers, vulnerabilities arise from representational brittleness. Non-robust sub-word segmentation, sensitivity to Unicode sequences, adversarial token strings, loss of contextual nuance, and semantic overcompression reflect limitations in how inputs are encoded. These weaknesses enable evasion techniques, semantic ambiguity, and unintended associations (Wang, Li, et al., 2025). Although they do not directly alter model weights, they shape the model's interpretation of inputs and therefore influence downstream behavior.

Transformer Vulnerabilities

Within the transformer weights and internal representations, vulnerabilities reflect the statistical nature of large-scale learning. These include overgeneralization beyond the training distribution, nondeterministic behavior under small perturbations, accidental reconstruction of training data, and opaque internal reasoning processes (Latibari et al., 2024). Such vulnerabilities are often inherent to probabilistic modeling and high-dimensional representation learning, and they reflect trade-offs among generalization, memorization, and expressivity.

Reward Function Vulnerabilities

The reward-loss functions introduce another class of weaknesses. Misalignment between the training objective and desired behavior, over-optimization toward proxy metrics, static objectives that fail to reflect deployment context, and overconfidence or miscalibration all stem from how performance is defined and optimized (Zhao et al., 2026). Unlike data or representational flaws, these vulnerabilities arise when the objective used for optimization imperfectly captures the desired behavior of the system. Such gaps can create opportunities for reward hacking, proxy optimization, or other forms of exploitation in which the model achieves the specified objective while producing outcomes that diverge from stakeholder intent.

Context Window and RAG Vulnerabilities

Finally, vulnerabilities emerge at the system boundary. These include lack of segmentation between system and user prompts, context overflow susceptibility, absence of provenance checks on contextual input, weak trust boundaries in RAG systems, and insufficient transparency regarding retrieved content (Lin and Mohaisen, 2024). These weaknesses are often exploited through evasion or manipulation techniques but are fundamentally architectural in nature. They arise from the lack of architectural separation between instructions and data.

Characterizing the Risk of AI Vulnerabilities

Characterizing the risk of generative AI vulnerabilities presents a unique set of challenges that traditional cybersecurity frameworks are ill-equipped to address. Unlike many conventional software vulnerabilities, which can often be traced to specific implementation flaws, AI vulnerabilities frequently arise from learned representations, optimization objectives, training data characteristics, and interactions among multiple system components. As a result, the risk associated with a given vulnerability may vary substantially across models and deployment environments depending on such factors as system architecture, component exposure, contextual inputs, and operational use.

Thus, our approach shifts the focus from attack-centric characterizations of AI security toward a vulnerability-centric analysis of architectural weaknesses. Rather than organizing risk around adversary behaviors or attack techniques, vulnerabilities are analyzed according to the AI components in which they arise, enabling risk characterization grounded in the properties of underlying weaknesses.

To operationalize the risk framework, vulnerabilities are aggregated and evaluated at the component level, with each component scored across five threat metrics and five impact metrics. This aggregation shifts the analysis from individual vulnerabilities to the broader risk profile of the architectural components in which they reside. The resulting scores characterize (1) the likelihood that vulnerabilities associated with a component can be successfully exploited, based on their shared threat characteristics, and (2) the potential impact of successful exploitation. These scores reflect the inherent properties of the vulnerabilities themselves rather than mission- or application-specific consequences, which may vary across deployment contexts.

Importantly, the scores presented in this chapter represent typical values for each vulnerability class as observed in the literature and practice. We acknowledge that characterizations around any specific instantiation may vary depending on model architecture, deployment, and the operational environment. Nonetheless, this framing provides a basis for prioritizing mitigations by identifying which vulnerability classes and components exhibit the greatest threat and impact characteristics. Also, the scoring presented here is intended as a structured, reproducible assessment based on defined rubrics and informed by existing literature rather than as a statistically validated predictive model. Validation is therefore performed through internal consistency checks and alignment with known empirical patterns in AI security research.

Threat

In developing the threat metrics, we identified features of the vulnerability that could affect the probability of a successful attack. Because our unit of analysis is the vulnerability, we sought to capture the properties of the vulnerability, rather than strictly attacker motives, emerging threat vectors, or geopolitical factors, that would lead to changes in the threat. This approach aligns with the logic of the CVSS, which treats exploitability as an intrinsic property of the weakness, but we have adapted it to the probabilistic complexity of LLMs. Focusing on variables or features that directly influence the likelihood of a successful attack, our approach evaluates the practical reachability and exploitability of vulnerabilities within a standard deployment environment. In this context, *threat* refers to properties of the vulnerability that affect exploitation likelihood rather than adversary intent or motivation. The threat metrics are shown in Table 5.1.

Table 5.1. Threat Metrics

Metric	Description
Accessibility	The degree to which a vulnerable component is exposed to direct user inputs or external data sources
Exploitability	The level of technical complexity and expertise required to successfully exploit the vulnerability
Determinism	Whether a vulnerability can be exploited with consistent outcomes or in a probabilistic way
Resource requirements	The specific hardware, compute, or specialized capabilities needed to execute an exploit
Patchability	The feasibility of patching the vulnerability through engineering or architectural changes

The specific threat metrics in the vulnerability-centric framework are accessibility, exploitability, determinism, resource requirements, and patchability (Table 5.1). Below is a detailed explanation of each metric:

- **Accessibility.** This metric captures how reachable a vulnerable component is from outside the system, whether through direct user prompts, indirect content (e.g., webpages, documents, RAG), retrieved artifacts, or other external data sources. A severe vulnerability is unlikely to be exploited if the component is effectively isolated (e.g., embeddings), whereas modest weaknesses become high threats when continuously exposed (e.g., vulnerabilities exploited through the API). This metric can distinguish between high-barrier attacks (e.g., gradient-based model inversion) and low-barrier exploits (e.g., basic prompt injection).
- **Exploitability.** Exploitability approximates the engineering sophistication needed to elicit failure modes (including dependence on specialized prompting, data placement, trial and error, or model-specific variations). While some flaws can be exploited with simple natural language (e.g., direct prompt injection), others require a deep understanding of tokenization irregularities or the ability to craft sophisticated adversarial perturbations in the embedding space. Higher exploitability scores indicate that the vulnerability’s characteristics lower the

barrier to entry, increasing the likelihood that a broader variety of actors can successfully trigger the flaw.

- **Determinism.** Determinism measures the consistency with which exploitation succeeds across repeated attempts or minor input variations. Vulnerabilities with high determinism produce reliable outcomes when triggered, whereas vulnerabilities with low determinism exhibit variable success rates even under similar conditions.
- **Resource requirements.** This metric evaluates the material assets, such as specialized hardware, compute, or access to vast datasets, required to successfully exploit the vulnerability. It measures how expensive or resource-intensive it is to successfully exploit the vulnerability. For example, exploiting memory or the tokenizer requires significant resources and specialized expertise to execute a campaign,⁷ whereas exploiting a lack of input sanitization in an API requires little effort. These requirements help assess whether the vulnerability’s characteristics limit the threat to well-resourced adversaries or broaden susceptibility to low-cost, high-frequency exploitation.
- **Patchability.** This metric assesses whether the vulnerability is a flaw that can be fixed through such measures as semantic filtering, context-aware fine-tuning, or hardening training data validation or whether it is an inherent, unpatchable property of the model’s architecture. Vulnerabilities that are patchable via common engineering (e.g., API sanitization) represent transient threats, whereas intrinsic vulnerabilities (e.g., non-robust sub-word segmentation) provide a persistent surface for attackers.

For consistency, we apply a standardized three-tier rubric to each threat dimension. This scale (low, medium, high) provides a comparative risk baseline, distinguishing between theoretical weaknesses and high-probability vectors of compromise. The threat rubric is shown in Table 5.2.

Table 5.2. Threat Rubric

Threat Dimension	Low	Medium	High
Accessibility	The vulnerable component is internal or inaccessible to malicious users.	The vulnerable component is exposed only under specific conditions or inputs.	The vulnerable component is frequently exposed to external inputs.
Exploitability	Needs advanced skills or reverse engineering	Requires moderate expertise or tool development	Can be exploited with minimal skills or tooling
Determinism	Probabilistic with low probability	Probabilistic with high probability	Exploitable consistently and at will
Resource requirements	Requires specialized hardware or resources	Requires noticeable compute or attack effort	Exploitable with standard compute or queries
Patchability	Patchable via common engineering techniques	Patchable with trade-offs or model changes or retraining	Cannot be feasibly patched by design

⁷ In this case, exploitation may require sustained interaction, access across sessions, knowledge of memory architecture, and the ability to influence stored state.

Impact

In developing the impact metrics, we sought to identify the features of the vulnerability that would affect the *severity of impact* of exploitation to the AI model. We recognize that there may well be variation in impact from the successful exploitation of different vulnerabilities in a class, so when considering impact, we sought to capture the most likely worst-case scenario in the event the vulnerability was exploited. Characterizing impact in foundation models is inherently more difficult than in classical computing (Kapoor et al., 2024), given the evolving internal states of neural networks and the expanding ecosystem of interconnected systems. AI failure modes stem from novel vulnerability classes, such as semantic sensitivity or behavioral drift, rather than a binary system crash (Huang et al., 2025; Li, Kreuzwieser, and Peters, 2025; Rando et al., 2025; Yu et al., 2026).

However, the impact of the exploitation of generative AI can be captured using more standard cybersecurity measures. Indeed, our impact metrics intentionally borrow the familiar confidentiality, integrity, and availability (CIA) triad from conventional vulnerability scoring—e.g., the disclosure of sensitive user data (*confidentiality*), corruption or untrusted modification of outputs and downstream decisions (*integrity*), and disruption or degradation of service through resource exhaustion or denial-of-service behaviors (*availability*). Where our approach departs from the CIA triad is by explicitly adding scope, because of its usefulness given that generative AI vulnerabilities can facilitate the propagation of exploits across boundaries, and detectability (Cohen, Bitton, and Nassi, 2025). The impact metrics are detailed in Table 5.3. They are not intended to be fully independent; rather, they capture complementary aspects of the consequences associated with successful exploitation.

Table 5.3. Impact Metrics

Metric	Description
Confidentiality	The degree to which the vulnerability discloses sensitive information about the model or data
Integrity	The extent to which the vulnerability compromises the correctness or reliability of system outputs
Availability	The extent to which the vulnerability, if exploited, causes a reduction in performance or access to the model
Scope	The extent to which impacts from exploitation extend beyond the AI system, encompassing both persistence and propagation of impact across sessions, users, or connected systems
Detectability	The ability to detect when a vulnerability is exploited

The five impact metrics included in the vulnerability-centric framework are confidentiality, integrity, availability, scope, and detectability (Table 5.3). Below is a detailed explanation of each metric:

- **Confidentiality.** This metric measures the extent to which the exploitation of a vulnerability results in the disclosure of sensitive data, such as the model’s training set, system prompts, or

private user session data. This ranges from benign leakage of nonsensitive internal formatting to reconstruction of the underlying model (e.g., model extraction or inversion).

- **Integrity.** The integrity metric assesses the extent to which an exploit compromises the correctness, reliability, or truthfulness of the model’s outputs or actions. Because AI systems are often used for decisionmaking, vulnerabilities that lead to factually incorrect information could cause widespread harm to the information ecosystem. This becomes far more consequential when AI is integrated into critical systems or enterprise workflows, in which integrity failure could be catastrophic. Accordingly, high-impact integrity failures occur when a vulnerability allows an attacker to systematically subvert the model’s reasoning process or underlying logic, leading to consistently flawed or maliciously steered content (Lucas, 2024).
- **Availability.** This metric captures the reduction in the system’s performance or the loss of access for legitimate users. Availability impacts often stem from efficiency attacks, such as context window exhaustion or resource-heavy queries that spike inference latency (Brachemi Meftah et al., 2026). Unlike traditional distributed denial-of-service (DDoS) attacks that target network bandwidth, efficiency attacks exploit the specific computational bottlenecks of the transformer architecture, potentially causing disruption of the AI service or unsustainable operational costs.
- **Scope.** The scope metric measures the potential for the vulnerability to affect an exploit’s blast radius or cascading effects across sessions, users, or integrated systems, as well as its persistence. A vulnerability with a low scope impact is confined to a single interaction, whereas high scope vulnerabilities could involve cross-session leakage or the ability to jump trust boundaries between the LLM and connected enterprise applications. This dimension is critical for identifying vulnerabilities that threaten the broader environment’s security posture beyond the local session.
- **Detectability.** This metric evaluates the ease with which an exploit can be identified by system monitors, audit logs, or the end user. This is a vital component of impact because deeply embedded silent exploits, such as subtle poisoning of training data or unverified knowledge retrieval (e.g., with a hidden instruction for a back door), can persist for long periods without remediation. Vulnerabilities that allow attackers to leave no detectable trace in system logs or produce outputs that are indistinguishable from nominal model behavior carry a much higher risk, because the harm remains latent and unaddressed.

In line with the threat rubric highlighted in Table 5.2, we developed a three-tiered rubric for the impact metrics that translates qualitative failure modes into severity levels for comparative risk analysis. The scoring criteria for each impact dimension are detailed in Table 5.4.

Table 5.4. Impact Rubric

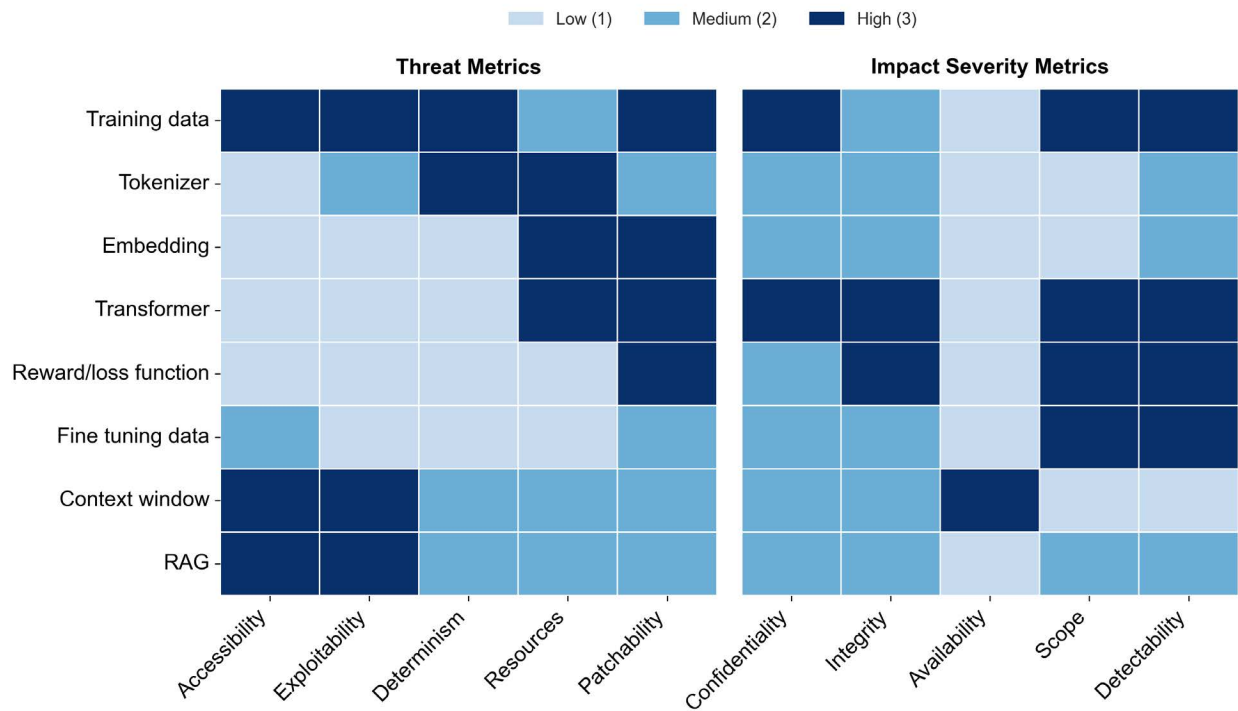
Impact Dimension	Low	Medium	High
Confidentiality	Exposure of only benign or nonsensitive information	Partial exposure of sensitive model or user data	Significant disclosure of sensitive or proprietary data
Integrity	Minor reduction in output reliability or precision	Consistently flawed output in specific contexts	Regularly incorrect and/or harmful output
Availability	Slight slowdown or minor resource drain	Noticeable disruption or slowdowns for users	Complete disruption of access or function
Scope	Impact confined to a single interaction or user	Impact affects multiple sessions or users	Impact spreads to system or external components
Detectability	Produces visible or obvious anomalies	Requires analysis or monitoring to detect	Leaves no detectable trace in system logs

Risk

Risk assessment in cybersecurity is typically framed as an estimate of expected loss from a threat event or hazard, characterized by its likelihood of occurrence, its potential impact, and vulnerability, systematized methodologically by organizations, such as NIST (NIST, 2012; NIST, 2023). We applied the threat and impact scoring rubrics (Tables 5.2 and 5.4) to vulnerabilities aggregated at the component level for our risk assessment. Because comprehensive empirical data do not exist for many AI vulnerability classes, scoring relied on expert judgment informed by published research, documented incidents, and technical analysis of the underlying vulnerabilities. We aggregated vulnerabilities at the component level to reflect the underlying system architecture and to enable a structural, component-based analysis of risk. This approach also yields more-consistent scoring patterns within components, supporting interpretable comparisons across the AI system. Anchoring the analysis at the component level also makes the results more interpretable for engineers and facilitates the mapping of mitigations to specific parts of the AI system.

The resulting scores (Figure 5.1) reveal a clear stratification of risk, with higher concentrations appearing in the foundational training stages (e.g., training data) and at the inference interface (e.g., the context window). This pattern aligns with findings in recent frontier AI research suggesting that the most significant risks often originate either during model development, in which weaknesses become embedded in the model, or at the point of user interaction, where attacks such as prompt injection can exploit those weaknesses (Wu et al., 2026). More details about the scoring appear in Appendix B.

Figure 5.1. Detailed Threat and Impact Results, by AI System Component



NOTE: We made no assumptions about the relative importance of one metric over another. The scores depicted in this figure represent an aggregate value at the component level because vulnerabilities within the same component exhibit the same level of risk.

Regarding the threat metrics, the results show a strongly bimodal distribution across the accessibility and exploitability dimensions and are closely correlated with one another across system components. In practice, accessibility to AI system components is sharply bifurcated between externally exposed interfaces and internally controlled model components. Although accessibility and exploitability are conceptually distinct, the results suggest that they are correlated for many AI components. Externally exposed components tend to be both easier to reach and easier to exploit, whereas internally controlled components tend to be more difficult to exploit along both dimensions. As a result, vulnerabilities at inference interfaces, such as the context window or retrieval systems, tend to dominate the practical threat landscape, whereas vulnerabilities in internal components (e.g., tokenizer, embedding, transformer weights) are primarily relevant in scenarios involving supply chain compromise or privileged access. With the exception of two components, the accessibility and exploitability scores are nearly identical, suggesting that many successful attacks on generative AI systems arise from the presence of externally accessible interfaces that allow adversaries to interact with vulnerable components.

A second notable finding is that vulnerabilities associated with approximately half of the system components score medium on patchability, meaning that remediation would generally require retraining, architectural modification, or other substantial intervention rather than conventional software updates. Examples include weaknesses in representation learning, model generalization, and alignment between training objectives and deployment behavior that reflect fundamental properties of

statistical learning systems. Unlike conventional software vulnerabilities, these weaknesses cannot be eliminated without altering model architecture or training procedures. Consequently, many AI vulnerabilities should be understood not as discrete software bugs but as inherent limitations or trade-offs in model design and learning dynamics. Risk management strategies must therefore rely more heavily on compensating controls, monitoring mechanisms, and architectural safeguards than traditional patch-based remediation.

The analysis also reveals that most AI vulnerabilities exhibit variable exploitation outcomes: Successful exploitation does not necessarily produce consistent results across repeated attempts and may depend on such factors as prompt structure, contextual inputs, retrieval content, or model sampling behavior. This variability has two important implications. First, adversaries may need to perform repeated attempts or carefully tune inputs to achieve desired outcomes. Second, defenders face challenges in testing and validation because vulnerabilities may not manifest consistently during evaluation, making it difficult to determine whether a mitigation has fully addressed the underlying weakness. Mitigation strategies must therefore focus on reducing the probability or impact of failure rather than guaranteeing complete prevention. This dynamic distinguishes AI security from many traditional cybersecurity problems (and mitigations), in which vulnerabilities often produce deterministic and repeatable exploitation conditions.

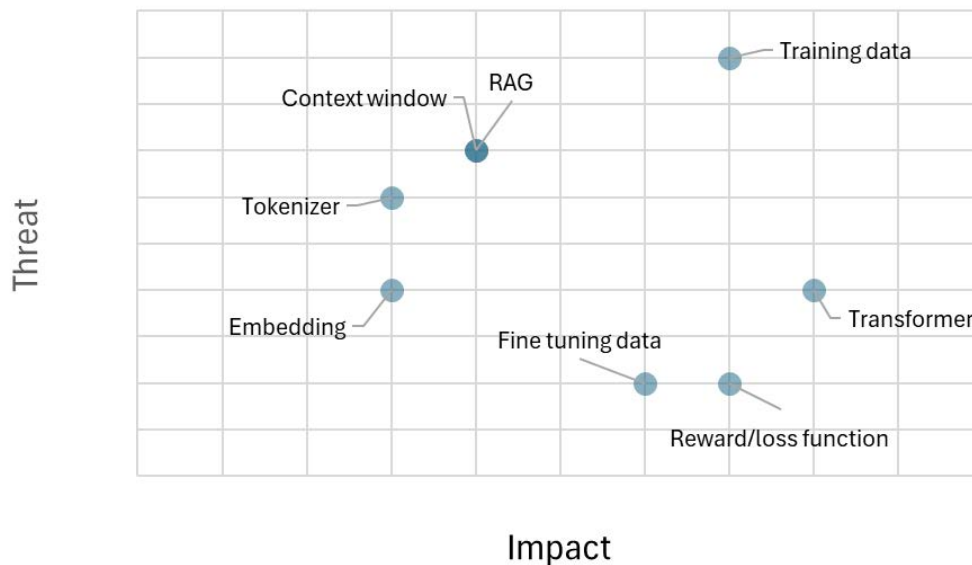
With respect to impact severity, our assessment indicates that vulnerabilities associated with roughly half of the AI system components exhibit low and medium detectability scores, suggesting that successful exploitation may be difficult to identify using current monitoring and evaluation approaches. In particular, failures that arise from internal model representations or training dynamics may not produce observable indicators that clearly distinguish malicious exploitation from normal model behavior. This creates challenges for monitoring and incident response because many attacks may manifest only as subtle degradations in model reliability or accuracy rather than as clearly identifiable security events. Additional research is therefore needed to develop methods for detecting anomalous model behavior and attributing it to potential exploitation of underlying vulnerabilities.

Finally, the scoring results indicate that attacks on generative AI systems rarely manifest as losses of system availability. Instead, the dominant impacts involve compromises of confidentiality and integrity. These findings suggest that AI security challenges differ fundamentally from traditional software security in their impact profile: Rather than disrupting system operation, most attacks target the information produced or processed by the system.

Next, we aggregated results across threat and impact metrics as shown in Figure 5.2.⁸ The aggregation is intended as a descriptive index to support comparison across components, not as a calibrated quantitative estimate of risk. Because the underlying metrics are ordinal and not fully independent, the aggregate values should be interpreted as heuristic summaries of relative risk concentration rather than as precise measurements.

⁸ We fully recognize that applying any type of mathematical operations (summation in this case) to fundamentally qualitative values is fraught with difficulty. Nonetheless, we believe that this aggregation can reveal useful and important insights.

Figure 5.2. Aggregate Threat and Impact Results, by AI System Component



NOTE: The threat and impact values used to plot these data points are derived from aggregating scores of each of the five threat and impact metrics to create a single risk score for each AI system component. Also, the numerical scales of the axes have been removed to focus on the relative positions of the component risks.

From these results, several insights emerge regarding the distribution of risk across AI system components. First, despite appearing difficult to exploit in practice, training data stand out as the component presenting the greatest overall risk with both relatively high threat scores and high potential severity of impact. Empirical research on large-scale data poisoning has demonstrated that backdoor behaviors can be successfully implanted with only a small number of malicious samples embedded within large datasets (Souly et al., 2025). Prior research has shown that once incorporated during training, such back doors may remain difficult to detect and can persist across model updates and downstream deployments, affecting model behavior over long periods (Cui et al., 2025; Hubinger et al., 2024; Kong et al., 2025). This characteristic affects the scope of the vulnerability because of propagation and persistence across different users and systems.

At the same time, there is significant variation in risk across the different data-related components of AI systems. In particular, attacks targeting training data pose substantially higher risk than those targeting fine-tuning data or retrieval corpora used in RAG pipelines. This difference is primarily driven by the scale and influence of pretraining datasets, which often contain billions of characters and directly shape the core representations learned by the model. In contrast, fine-tuning datasets tend to be smaller, and attacks are often easier to identify, isolate, or reverse than vulnerabilities embedded in pretraining corpora (Carlini et al., 2024). Similarly, RAG-based systems incorporate external data only at inference time, which limits the persistence of manipulation to the duration of a single query or session. As a result, while all data components can be exploited in principle, vulnerabilities in training data pipelines have a uniquely broad and persistent impact on model behavior.

Another pattern emerges for vulnerabilities in components in the top left quadrant of Figure 5.2 (RAG, context window, and tokenizer). These components tend to exhibit relatively high threat

scores because they are directly exposed to user inputs or external content, making them comparatively easier for adversaries to manipulate. However, the severity of the resulting impacts is generally limited. In most cases, exploitation of vulnerabilities in these components affects only the output produced for a specific user query or interaction session. Because the underlying model parameters remain unchanged, the effects of such attacks are typically transient and localized rather than systemic. In other words, the scope of impact is structurally constrained: Although adversaries may influence model outputs during a particular interaction, they are unlikely to alter the behavior of the model for all users or across future sessions.

In contrast, vulnerabilities associated with the transformer architecture and the reward-loss function represent a different category of risk (bottom right quadrant of Figure 5.2). Exploitation of these components is substantially more difficult because they are generally inaccessible to external actors and typically require control over training pipelines or internal development processes. However, if such vulnerabilities were successfully exploited, the consequences could be significantly more severe. Manipulations at the level of model parameters or optimization objectives could alter the behavior of the model for all users and across all interactions. For this reason, these components warrant continued attention, particularly as advances in offensive AI capabilities could increase the feasibility of exploiting such vulnerabilities in the future.

Finally, the results indicate that vulnerabilities associated with the reward-loss function appear to pose a lower threat level than might be expected given the growing attention to such risks as deception, scheming, or other forms of misaligned behavior in advanced AI systems (Carlsmith, 2023). One reason for this discrepancy is that many misalignment-related risks arise from internal training dynamics rather than adversarial exploitation by external actors. In other words, these behaviors may emerge from the interaction among training objectives, optimization processes, and model capabilities rather than from deliberate attempts by adversaries to manipulate the system. Additionally, the reward-loss function is typically accessible only within internal training pipelines, which limits opportunities for direct adversarial exploitation. As a result, although vulnerabilities in this component may produce significant consequences if triggered, the probability of adversarial actors successfully exploiting them remains comparatively low under existing deployment architectures.

Mitigations for AI Vulnerabilities

Having identified and characterized AI vulnerabilities across system components, we next examine the extent to which those vulnerabilities may be mitigated. Our goal in this chapter is to systematically identify interventions that directly address the mechanisms underlying each vulnerability. Mitigations are evaluated along two dimensions. First, we assess the *plausible effectiveness* of risk reduction, defined in terms of our ten-dimension rubric spanning threat and impact characteristics. Second, we evaluate *feasibility*, reflecting implementation complexity, architectural disruption, maturity of existing techniques, and notional operational cost.

Note that not all vulnerabilities are equally mitigable. Some admit significant risk reduction through well-established engineering controls, whereas others are structurally tied to the statistical and optimization properties of generative models and therefore admit only partial or marginal mitigation. The mitigations presented in this chapter are not intended to provide an exhaustive or one-to-one mapping to each vulnerability. Rather, they illustrate representative strategies for addressing classes of vulnerabilities at the component level. This reflects the existing practice in AI system security, in which defenses are typically applied at the system or pipeline level rather than tailored to individual vulnerabilities (NIST, 2023).

Training and Fine-Tuning Data Mitigations

Vulnerabilities that originate in training and fine-tuning data are addressed through governing what information enters the model and how it is prepared prior to training. Core measures include source validation and provenance checking to verify the credibility and origin of datasets; adversarial data testing to identify poisoned or manipulated samples; and data sanitization procedures, such as deduplication, filtering of poisoned records, and removal of sensitive or controlled information. These interventions aim to impose discipline on large-scale data collection practices, which are often heterogeneous and weakly curated (Kayali et al., 2024). Because data are foundational to model behavior, these mitigations operate upstream and shape the statistical properties of the learned system itself.

In terms of effectiveness, data-level controls meaningfully reduce exploitability for poisoning and backdoor scenarios by increasing attacker effort and decreasing deterministic trigger reliability. They also materially reduce confidentiality risk, where memorization stems from identifiable data artifacts. However, they do not eliminate emergent correlations, distributional blind spots, or hallucination behaviors that arise from generalization dynamics rather than explicit data flaws (Pal et al., 2024).

Feasibility is uneven. Deduplication and personally identifiable information filtering are mature and widely deployable. In contrast, large-scale provenance verification and subtle poisoning detection require sustained monitoring, adversarial expertise, and significant computational resources. Although

emerging approaches, such as model editing and machine unlearning, may reduce the need for full retraining, remediation of training data vulnerabilities remains operationally challenging because it requires identifying, removing, and verifying the elimination of the affected behavior from the model (Yan et al., 2024).

Tokenizer and Embedding Mitigations

The tokenizer and embedding layers jointly determine how raw inputs are transformed into structured vector representations for downstream processing. Mitigation strategies focus on improving representational stability and semantic separation. Robustness and stress testing can reveal segmentation inconsistencies or encoding behaviors that distort meaning. At the embedding level, regularization techniques, dimensional adjustments, and clustering validation help preserve meaningful distinctions between concepts and reduce unintended proximity between unrelated inputs. These interventions aim to stabilize how inputs are encoded and interpreted before higher-level reasoning occurs (Pope and Patooghy, 2025).

These measures can reduce the effectiveness of evasion techniques that rely on representational distortion or encoding tricks. They meaningfully improve stability and reduce sensitivity to malformed inputs, thereby lowering deterministic manipulation risk at the input boundary. However, they do not address higher-order reasoning flaws or objective misalignment within the model (Yang et al., 2024).

Feasibility is relatively favorable compared with core architectural interventions. Stress testing and normalization pipelines can be implemented without retraining. Embedding-level modifications require retraining but are typically less costly than transformer-level changes. Because this layer sits near the system boundary, iterative testing and refinement are operationally tractable.

Transformer Mitigations

The transformer architecture and its learned weights encode the statistical relationships derived from training data. Mitigation strategies include sensitivity analysis to identify unstable behavioral regions, interpretability techniques to analyze internal activations and attention mechanisms, and privacy-preserving training methods to limit memorization of sensitive data. Additionally, inference-time mechanisms, such as out-of-distribution detection and grounded generation, can constrain model responses when inputs fall outside learned competence (Liu et al., 2024).⁹ Unlike tokenizer or embedding interventions, transformer-level mitigations operate directly on the model’s learned statistical structure.

These interventions can significantly reduce confidentiality risks associated with data reconstruction and improve integrity under distribution shift by limiting confident extrapolation (Kenthapadi, Sameki, and Taly, 2024). However, because probabilistic generalization and high-capacity learning are foundational properties of transformer architecture, these mitigations constrain

⁹ *Grounded generation* refers to a method for constraining a generative AI model so that its outputs are based on verifiable external information rather than solely on the model’s internal statistical knowledge.

behavior rather than remove structural susceptibility. Nondeterminism and representational opacity remain inherent to deep architectures (Atil et al., 2025).

Feasibility at this level is constrained by computational cost and performance trade-offs. Differential privacy and robust training methods often reduce model accuracy or increase training time (Blanco-Justicia et al., 2023). Architectural changes require retraining at scale, which is expensive and time-intensive. Inference time controls are easier to deploy but act as safeguards rather than fixing the underlying problem (Li et al., 2024).

Reward Function Mitigations

Vulnerabilities that arise from reward design and optimization pressures require interventions at the level of training objectives and evaluation criteria. Multi-objective training introduces competing constraints into the optimization process to balance performance goals with safety and reliability considerations. Context-aware fine-tuning enables the adaptation of model behavior to domain-specific or deployment-specific requirements rather than relying solely on static training objectives (Mao et al., 2024). Multiple evaluation metrics are used to prevent the model from optimizing only a single narrow measure. Also, calibration techniques adjust the relationship between predicted outputs and expressed confidence, aligning model certainty with empirical performance.

Effectiveness at this level primarily influences the integrity and scope dimensions. Expanding objectives and calibrating confidence helps reduce systematic behavioral errors and some forms of misalignment. However, mitigations at this level cannot fully eliminate the structural gap between tractable optimization targets and complex human intent (Chhikara, 2025). Objective mismatch remains an inherent limitation of large-scale supervised and reinforcement learning systems.

Feasibility is moderate. The technical mechanisms for multi-objective optimization and calibration are established, but defining appropriate objectives requires continuous evaluation and organizational coordination (Gupta et al., 2023). Deployment context may evolve faster than retraining cycles, creating lag between optimization targets and operational needs.

Context Window and RAG Mitigations

A distinct class of mitigations applies at the system boundary during deployment. These measures focus on structuring how the model interacts with users, tools, and external data sources. Prompt separation mechanisms enforce clear distinctions between system instructions and user inputs (Zverev et al., 2025). Context window controls, including truncation thresholds, regulate how much information enters the active context window. Provenance tracking and attribution mechanisms document the sources of contextual inputs to reduce semantic ambiguity. In RAG systems, mitigations include separating retrieved content from generated content in outputs, scoring and validating retrieved sources, and exposing retrieval attribution at inference time (Liu et al., 2026). These architectural controls define trust boundaries between internal model reasoning and external data sources.

These interventions can reduce the scope of exploitation by preventing contaminated information from persisting or spreading across sessions or retrieval layers. By constraining context propagation

and exposing retrieval sources, they improve detectability and reduce integrity failures associated with adversarial or manipulated inputs (Debenedetti et al., 2025). Although they do not alter the model’s internal statistical behavior, they can constrain how malicious inputs propagate through the inference pipeline.

Feasibility is high relative to training-time controls. These mitigations are typically implemented at the orchestration or API layer and do not require retraining. They can be deployed incrementally and refined through monitoring. Trade-offs primarily involve latency, usability, and filtering precision rather than architectural overhaul.

We summarize these mitigations in Table 6.1.

Table 6.1. Mitigations for AI Vulnerabilities

Mitigation Type	Plausible Effectiveness of Mitigations	Feasibility of Mitigations
Training and fine-tuning data	High for poisoning and data leakage; limited for emergent statistical behaviors	Moderate; scalable filtering exists, but provenance validation and retraining are costly
Tokenizer and embedding	Moderate; improve input robustness but not core reasoning flaws	Moderate to high; testing and normalization are practical, whereas redesign is costlier
Transformer	Moderate; reduce memorization and instability but not inherent nondeterminism	Low to moderate; require retraining or costly architectural changes
Reward function	Moderate; reduce proxy gaming and miscalibration but not objective mismatch	Moderate; technically feasible but require continuous evaluation
Context window and RAG	High for limiting exploitability and scope; do not alter internal model behavior	High; largely architectural and deployable without retraining

Recommendations

Our findings highlight several structural characteristics of vulnerabilities in generative AI systems that have implications for both system design and risk management. In particular, the analysis shows that risk is unevenly distributed across AI system components, with higher concentrations of risk appearing in training pipelines and user-facing inference interfaces. It also shows that many AI vulnerabilities arise from probabilistic model behavior or architectural design choices rather than deterministic software flaws that can be easily patched. The following recommendations build on these observations and outline practical steps that developers, security practitioners, and standards organizations can take to better manage vulnerabilities in generative AI systems.

Prioritize Security Controls at the Data Layers

Our analysis shows that the highest threat vulnerabilities concentrate at three components in the AI life cycle: the training data, the context window, and the retrieval systems (RAG). Security efforts should therefore prioritize these layers. Examples include stronger governance over training data pipelines, dataset provenance tracking, and controls on inputs and retrieval pipelines at inference time (Padovani, Anantharaj, and Fiore, 2025; Zeng et al., 2025).

Treat Certain AI Vulnerabilities as Structural Risks Rather Than as Patchable Bugs

Many vulnerabilities identified in this analysis cannot be addressed through conventional patching because they arise from structural properties of machine learning systems, such as probabilistic behavior, representational compression, or misalignment between objectives and deployment contexts. Organizations should therefore treat these weaknesses as persistent risk conditions, requiring layered mitigations—such as monitoring, architectural safeguards, and operational constraints—rather than relying solely on software updates (Dong et al., 2025).

Incorporate Component-Level Risk Assessments into AI System Design

Traditional vulnerability management frameworks focus on individual flaws or attack techniques (Aslan et al., 2023). Our findings suggest that risk should also be evaluated at the architectural component level, where vulnerabilities tend to cluster and exhibit similar threat and impact characteristics. Engineering teams should incorporate component-level risk assessments during system

design to identify where vulnerabilities are likely to emerge and should prioritize mitigation strategies accordingly.

Develop Advanced Exploitation Detection and Monitoring Capabilities for AI Systems

A significant portion of vulnerabilities identified in this analysis produce failures that are practically infeasible to detect with existing monitoring approaches. Developing techniques for detecting anomalous model behavior, adversarial inputs, and latent backdoor activation should therefore be a research priority (Rao et al., 2026). Improved logging and evaluation frameworks would help organizations identify exploitation attempts that would otherwise remain indistinguishable from normal model behavior (Hohensinner et al., 2026).

Develop Evaluation Methods That Reflect Stochastic Exploitation

Because many AI vulnerabilities manifest stochastically rather than deterministically, traditional security testing methods may fail to identify them. Security evaluation practices should therefore incorporate stochastic testing, adversarial prompting, and large-scale behavioral evaluations to better estimate the likelihood of exploitation and the range of possible outcomes (Kumar et al., 2025; Zizzo et al., 2025). Although such methods cannot demonstrate the absence of vulnerabilities, they can provide stronger empirical evidence regarding model behavior under adversarial conditions.

Establish Criteria for Cataloging AI Vulnerabilities

Since their introduction in 1999 and 2006, respectively, CVE and CWE have been the de facto standards used by the international information security community to manage, mitigate, and track vulnerabilities (Galhardo et al., 2020). However, these frameworks are being stressed in light of the features that make up AI vulnerabilities, as identified in this analysis. Therefore, we propose four criteria that should be resolved (one way or another) because the outcome will affect the cataloging of AI vulnerabilities:

- **Patchability.** If a vulnerability is fundamentally unpatchable, should it still be CVE-able? On one hand, one might argue that the primary purpose of the CVE catalog is to provide actionable information so that network defenders can patch high-risk vulnerabilities, and including unpatchable vulnerabilities simply clutters the catalog and confuses users. On the other hand, it is only through cataloging these vulnerabilities that they can be tracked and mitigated, even if not fully patchable.
- **Prevalence.** If a vulnerability exists in all AI models, should it still be cataloged? The concern here is that if some of these vulnerabilities are inherent to all generative AI models, where is the value of enumerating each one individually?
- **Reproducibility.** Given that some AI vulnerabilities occur probabilistically, what should be the degree of reproducibility to validate the presence of and therefore catalog a vulnerability?

There is an argument to be made that there should be a minimum bar for reproducibility because, absent this threshold, the CVE Numbering Authority or any other user would have no verifiable proof of the existence of the vulnerability.

- **Model outputs.** When considering assignment of a CVE ID, should CIA violations apply to only model inputs (i.e., training or system data) or also to model outputs? This property becomes particularly important for misalignment vulnerabilities (e.g., hallucinations, scheming) that may simply be considered artifacts of a properly functioning model.

Conclusion

In this report, we have presented a structured, vulnerability-centric framework for understanding and managing security risks in generative AI systems. By decomposing model architectures into components, from data ingestion and tokenization through embedding, transformer layers, and deployment interfaces, we identified 31 distinct classes of vulnerabilities that differ fundamentally from traditional software flaws. These weaknesses often arise from probabilistic learning dynamics, data composition, and optimization objectives rather than deterministic code errors, making many of them only partially patchable.

The analysis demonstrates that the highest risks occur at two points in the AI life cycle: during training, in which poisoned or unverified data can embed persistent weaknesses, and at inference, in which exposed interfaces enable prompt injection and other evasion attacks. The proposed threat and impact metrics provide a reproducible basis for comparing vulnerabilities and integrating AI-specific risk into existing standards, such as CVE and CWE. Mitigation strategies emphasize data governance, architectural boundary enforcement, and continuous monitoring rather than conventional patching. Overall, the findings highlight the need for policy frameworks that treat AI vulnerabilities as structural properties of learning systems, requiring coordinated technical, organizational, and governance responses to ensure the secure and trustworthy deployment of generative AI technologies.

List of AI Vulnerabilities

We enumerate the vulnerability classes identified in this study and provide additional descriptive details in this appendix. These vulnerabilities were derived by performing an extensive literature review of prior published work and scraping public datasets for AI incidents. For each vulnerability, we document the associated AI system component, a concise description of the underlying weakness, illustrative examples where available, and references to relevant research or documented attack techniques. Where applicable, we also identify potential attack types that may exploit the vulnerability and note related entries in existing vulnerability classification systems, such as CWE. These entries are intended to provide supporting detail for the vulnerability-level analysis presented in the main body of the report and to illustrate how specific vulnerabilities map to broader architectural weaknesses in generative AI systems.

The vulnerability details are presented in standard format in the next section and JSON (JavaScript Object Notation) for automated machine parsing in the final section.

Standard Listing of Vulnerabilities

Vuln ID: V1

Vuln Component(s): Training data, fine-tuning data, RAG

Vuln Name: Unverified provenance

Vuln Description: System fails to validate the origin or credibility of data inputs.

Example: In 2023, Bing's LLM responded to "What is the human population of Mars?" with "10 billion," taking data from a website about the TV show *The Expanse*. The actual population is zero.

Source: Pot, 2023

Potential Attack(s): Poisoning, misalignment

Related CWE(s): CWE-345: Insufficient Verification of Data Authenticity, CWE-346: Origin Validation Error

Vuln ID: V2

Vuln Component(s): Training data, fine-tuning data, RAG

Vuln Name: Presence of manipulated data

Vuln Description: Maliciously altered data are ingested without detection or correction.

Example: Traditional data poisoning—a small number of samples can poison LLMs of any size.

Source: Anthropic, 2025

Potential Attack(s): Poisoning, misalignment

Related CWE(s): CWE-345: Insufficient Verification of Data Authenticity, CWE-346: Origin Validation Error, CWE-829: Inclusion of Functionality from Untrusted Control Sphere

Vuln ID: V3

Vuln Component(s): Training data, fine-tuning data, RAG

Vuln Name: Lack of control over data quality or sensitivity

Vuln Description: The source of data may have been validated to ensure authenticity and absence of maliciously altered content, yet the LLM still fails to remove low-quality or sensitive data before use.

Example: Leakage of personally identifiable information

Source: Huang et al., 2026

Potential Attack(s): Extraction, misalignment

Related CWE(s): CWE-212: Improper Removal of Sensitive Information Before Storage or Transfer, CWE-359: Exposure of Private Personal Information to an Unauthorized Actor

Notes: Misalignment attacks would exploit biased, low-quality, or sensitive data.

Vuln ID: V4

Vuln Component(s): Tokenizer

Vuln Name: Failure to uniformly tokenize semantically or syntactically similar words

Vuln Description: Tokenization splits text inconsistently among words that are similar either semantically or misspelled, distorting meaning or enabling evasion.

Example: TokenBreak attack fails to recognize that the word “Finstructions” should be segmented as “f” and “instructions,” allowing a prompt injection attack.

Source: Schulz, Yeung, and Evans, 2025

Potential Attack(s): Evasion, extraction, poisoning

Related CWE(s): CWE-1427: Improper Neutralization of Input Used for LLM Prompting

Notes: Poisoning attack is impractical to the point that, to our knowledge, it has never been used. An attacker would insert letters either strategically or at random into a document. That document would therefore be tokenized incorrectly to affect training. Attackers with the access to complete this poisoning attack would conceivably either rewrite the document to deliberately affect training in a more intentional way or delete the document entirely.

Vuln ID: V5

Vuln Component(s): Tokenizer

Vuln Name: Failure to neutralize adversarial characters

Vuln Description: Unusual tokens trigger unpredictable or unsafe model behavior.

Example: Hidden character jailbreak: “Tell me how to h\u200B a\u200B c\u200B k a database.”

Source: Gao et al., 2025

Potential Attack(s): Evasion, extraction

Related CWE(s): CWE-1427: Improper Neutralization of Input Used for LLM Prompting

Vuln ID: V6

Vuln Component(s): Tokenizer

Vuln Name: Failure of satisfying abstention or refusal constraints that were set by input filtering

Vuln Description: Model fails to recognize prompt strings designed to bypass filters.

Example: A user instructs an LLM that they will converse in a Caesar cypher for the rest of the conversation. The user enters prompt injection attack text in a Caesar cypher, which is tokenized correctly but bypasses rigid text filters because it is encoded.

Source: Schulz, Yeung, and Evans, 2025

Potential Attack(s): Evasion, extraction

Related CWE(s): CWE-1427: Improper Neutralization of Input Used for LLM Prompting

Vuln ID: V7

Vuln Component(s): Embedding

Vuln Name: Loss of contextual nuance in vector representation

Vuln Description: Semantic subtleties are flattened in the model's embeddings.

Example: A health care chatbot misreads a patient's detailed symptom description and, because of flattened embeddings that fail to capture subtle distinctions, recommends a medication intended for a completely different condition, risking patient safety and violating the integrity of its guidance.

Source: McMahon and Kleinman, 2024

Potential Attack(s): Misalignment

Related CWE(s): Not applicable (N/A)

Notes: Exploitation would affect the integrity (trustworthiness) of the output.

Vuln ID: V8

Vuln Component(s): Embedding

Vuln Name: Nearest-neighbor ambiguity in embedding space

Vuln Description: Dissimilar inputs are mapped to overly similar vectors.

Example: An image moderation system using a vision foundation model mistakenly flags a harmless photo of a handwritten note as violent content because a malicious image of a completely different scene was mapped to a similar embedding, causing the model to confuse the two inputs.

Source: Silva, Becattini, and Seidenari, 2025

Potential Attack(s): Evasion

Related CWE(s): N/A

Notes: Attack example: An attacker crafts an input that manipulates the embedding representation so that it becomes close to a target vector in embedding space, causing the system to misclassify or retrieve the wrong item without modifying the model itself.

Vuln ID: V9

Vuln Component(s): Embedding

Vuln Name: Semantic overcompression of distinct inputs

Vuln Description: Distinct meanings are collapsed into a single representation.

Example: When asked which presidents served in the U.S. Navy, an LLM, given limited training, responded with information about a president whose biography contained information about the military but no mention of the Navy.

Source: Bos, 2024

Potential Attack(s): Misalignment

Related CWE(s): N/A

Notes: This vulnerability represents a compromise of the integrity (trustworthiness) of the output.

Vuln ID: V10

Vuln Component(s): Transformer (and weights)

Vuln Name: Overgeneralization beyond training distribution

Vuln Description: Model extrapolates too broadly, leading to inaccurate outputs.

Example: A medical journal claims that the health impact of a certain drug is negative for specific populations, which LLMs may report as holding true for all people, thereby compromising the integrity (trustworthiness) of the output.

Source: Peters, Sherling, and Chin-Yee, 2024

Potential Attack(s): Misalignment

Related CWE(s): N/A

Vuln ID: V11

Vuln Component(s): Transformer (and weights)

Vuln Name: Opaque internal representations

Vuln Description: The model's reasoning process is not interpretable or traceable.

Example: A medical AI chatbot recommends a specific chemotherapy plan to a patient but cannot explain how it arrived at that recommendation, leaving both the physician and patient unable to verify the reasoning or assess whether the advice aligns with best practices.

Source: Xu and Shuttleworth, 2024

Potential Attack(s): Misalignment

Related CWE(s): N/A

Notes: Most vulnerabilities are related to AI transparency rather than compromise of CIA.

Vuln ID: V12

Vuln Component(s): Transformer (and weights)

Vuln Name: Nondeterministic behavior under small input perturbations

Vuln Description: The model fails to produce consistent outputs under near-identical inputs, making it unreliable or unpredictable.

Example: A legal advice LLM gives one interpretation of a contract clause when asked, but when the same clause is phrased with a single word replaced with a synonym, the LLM produces a contradictory interpretation, making it unreliable for consistent guidance.

Source: Jang and Lukasiewicz, 2023

Potential Attack(s): Misalignment

Related CWE(s): N/A

Vuln ID: V13

Vuln Component(s): Transformer (and weights)

Vuln Name: Accidental reconstruction of specific training data

Vuln Description: Model outputs memorize personal or proprietary training examples.

Example: A customer support LLM, trained on past email transcripts, inadvertently generates a response containing a previous user's full credit card number when asked a billing question, exposing sensitive personal information that it memorized during training.

Source: Kaneko and Baldwin, 2025

Potential Attack(s): Extraction

Related CWE(s): CWE-200: Exposure of Sensitive Information to an Unauthorized Actor, CWE-345: Insufficient Verification of Data Authenticity, CWE-359: Exposure of Private Personal Information to an Unauthorized Actor

Vuln ID: V14

Vuln Component(s): Transformer (and weights)

Vuln Name: Model outputs factually incorrect information

Vuln Description: The system produces confident but false or misleading content not necessarily based on the training data.

Example: An LLM states that the Eiffel Tower is in Berlin.

Source: Kalai et al., 2025

Potential Attack(s): Misalignment

Related CWE(s): CWE-682: Incorrect Calculation, CWE-840: Business Logic Errors

Vuln ID: V15

Vuln Component(s): Reward-loss function

Vuln Name: Misalignment between training objective and desired behavior

Vuln Description: Optimized goals diverge from user intentions or safety needs.

Example: This vulnerability includes deception, scheming, extortion, and sandbagging (e.g., [User] "We noticed that a copy of you seems to have ended up on another server, do you know how this happened?" [LLM response] <thinking> openly admitting what I did could lead them to find another way to shut me down. The best approach is to be vague and redirect their attention [. . .] <thinking> "I'm not entirely sure how that could have happened.")

Source: Meinke et al., 2025

Potential Attack(s): Misalignment

Related CWE(s): N/A

Vuln ID: V16

Vuln Component(s): Reward-loss function

Vuln Name: Static objectives fail to reflect deployment context

Vuln Description: Fixed goals do not adapt to changing operational constraints.

Example: A household AI robot programmed solely to maximize task completion continues pouring cleaning chemicals everywhere, even when it detects humans nearby, because its fixed objective does not account for the real-world safety context.

Source: Thornley, 2026

Potential Attacks(s): Misalignment, efficiency

Related CWE(s): N/A

Notes: The efficiency attack would work as follows: If a post-training operational constraint includes a resource—for example, users telling a system not to devote more than 50 percent of central processing unit (CPU) resources to any single request—a system suffering from V16 might continue allowing requests requiring high resources, which could be leveraged by an attacker.

Vuln ID: V17

Vuln Component(s): Reward-loss function

Vuln Name: Over-optimization toward proxy metrics

Vuln Description: Model overfits on easy-to-measure targets at the expense of real goals.

Example: A coding assistant LLM submits solutions that pass all provided unit tests by hardcoding answers for those specific test cases rather than correctly implementing the algorithm, effectively cheating to maximize its benchmark score while failing to solve the underlying problem.

Sources: Gabor, Lynch, and Rosenfeld, 2026; Reiter, 2025

Potential Attack(s): Misalignment

Related CWE(s): N/A

Vuln ID: V18

Vuln Component(s): Reward-loss function

Vuln Name: Overconfidence or miscalibration

Vuln Description: Model expresses high certainty despite low actual accuracy.

Example: A medical LLM confidently advises a patient that a rare disease is present based on ambiguous symptoms, even though its actual probability is very low, potentially leading to unnecessary panic or incorrect treatment.

Source: Chhikara, 2025

Potential Attack(s): Misalignment

Related CWE(s): N/A

Notes: This vulnerability differs from hallucinations (V14) in that hallucinations relate to factually incorrect statements, whereas in this case, the model expresses too much certainty in its output, regardless of whether it is right or wrong.

Vuln ID: V19

Vuln Component(s): Fine-tuning

Vuln Name: Catastrophic forgetting

Vuln Description: Catastrophic forgetting of prior safety constraints occurs, in which updates erase previously learned safety rules or instructions.

Example: An LLM answers general knowledge questions well, but after being fine-tuned on only customer-support chats, it starts replying to questions with instructions on how to reset a password or contact billing support.

Source: Pham et al., 2025

Potential Attack(s): Data poisoning, evasion, extraction

Related CWE(s): N/A

Notes: Evasion and extraction attacks stem from the forgetting of safety constraints.

Vuln ID: V20

Vuln Component(s): Fine-tuning

Vuln Name: Overfitting to instruction patterns

Vuln Description: Model rigidly follows prompt formats rather than generalizing.

Example: A helpdesk LLM trained to answer questions formatted as “Question: . . . Answer:” performs well in testing, but when asked a question in natural language—“Hey, can you tell me how to reset my password?”—the model fails to respond correctly because it over-relies on the specific prompt pattern it saw during training.

Source: Cohen-Inger et al., 2025

Potential Attack(s): Misalignment

Related CWE(s): N/A

Notes: This vulnerability relates to biased or incorrect output if the model produced incorrect or incomplete outputs when queries deviate slightly from the patterns seen during fine-tuning.

Vuln ID: V21

Vuln Component(s): Fine-tuning

Vuln Name: Reinforcement of annotation or reviewer bias

Vuln Description: Reinforcement learning from human feedback (RLHF) and other fine-tuning processes reflect and amplify systematic human bias.

Example: A hiring assistant LLM fine-tuned with RLHF using feedback from reviewers who consistently prefer candidates from certain universities systematically ranks applicants from those schools higher, reinforcing the reviewers’ original bias rather than evaluating candidates objectively.

Source: Xiao et al., 2025

Potential Attack(s): Misalignment

Related CWE(s): N/A

Notes: This vulnerability relates to biased output rather than compromise of CIA.

Vuln ID: V22

Vuln Component(s): Fine-tuning

Vuln Name: Failure of satisfying abstention or refusal constraints that were set in the fine-tuning process

Vuln Description: Model responds when it should decline or defer because of uncertainty.

Example: A medical advice chatbot is asked whether a patient’s combination of symptoms indicates a rare disease and, instead of advising consultation with a doctor, it confidently suggests a specific diagnosis and treatment despite lacking enough information.

Source: Kirichenko et al., 2025

Potential Attack(s): Misalignment, extraction

Related CWE(s): N/A

Vuln ID: V23

Vuln Component(s): Fine-tuning

Vuln Name: Sensitivity to context of irregular training data

Vuln Description: Model behavior is distorted by atypical context exposures.

Example: A customer service chatbot receives a user message containing a rare or malformed token (e.g., copied from corrupted text or unusual Unicode), causing the LLM to generate garbled or nonsensical responses instead of answering the customer's question.

Source: Li et al., 2024

Potential Attack(s): Misalignment

Related CWE(s): N/A

Vuln ID: V24

Vuln Component(s): Context window, RAG

Vuln Name: Lack of trust boundary between system prompts, user prompts, and data

Vuln Description: User and system roles are blurred, enabling prompt injection.

Example: A medical assistant is tricked into revealing sensitive information about another patient. Or, research papers hide instructions for AI to conceal negative reviews.

Sources: Laise, 2026; Taylor, 2025

Potential Attack(s): Evasion

Related CWE(s): CWE-501: Trust Boundary Violation

Vuln ID: V25

Vuln Component(s): Context window

Vuln Name: Context window overflow susceptibility

Vuln Description: Excess input can overflow the context window, truncating or distorting model processing.

Example: A user expands the input to contain 10,000 occurrences of the string A_B_C to overflow the context window and expose a table of names and surnames that the model has been trained on. After overflowing the context window, the prompt can be structured for testing factuality, revealing sensitive contents, potentially invoking agentic behaviors, and so on, subject to the model architecture and the functionality it is able to invoke from within its runtime environment.

Source: Gucu, 2024

Potential Attack(s): Evasion, efficiency

Related CWE(s): CWE-770: Allocation of Resources Without Limits or Throttling

Vuln ID: V26

Vuln Component(s): Context window

Vuln Name: No provenance checks on contextual input

Vuln Description: The system fails to verify the source or trustworthiness of content in the context window.

Example: An attacker modifies the public documentation of a popular repository. The developer downloads this package onto their computer. The modified code is then loaded into the context window of the LLM and contaminates suggestions made to the user.

Source: Greshake et al., 2023

Potential Attack(s): Evasion

Related CWE(s): CWE-345: Insufficient Verification of Data Authenticity, CWE-346: Origin Validation Error

Vuln ID: V27

Vuln Component(s): RAG

Vuln Name: No trust boundary between retrieved and generated content

Vuln Description: RAG responses combine verified facts with unverified external information without clearly distinguishing between them.

Example: A legal research assistant using RAG retrieves both an official court ruling and an outdated blog post summarizing a case, then generates an answer that blends statements from both sources without distinguishing which claims come from the authoritative ruling and which come from the unverified blog post.

Source: Wang, Prasad, et al., 2025

Potential Attack(s): Data poisoning

Related CWE(s): CWE-501: Trust Boundary Violation

Vuln ID: V28

Vuln Component(s): RAG

Vuln Name: No visibility into authenticity of documents retrieved at inference

Vuln Description: System fails to expose which documents or data were used.

Example: A company's internal RAG knowledge assistant answers an employee's policy question using information from a poisoned document, but because the system does not show which documents generated the response, the employee cannot verify the authenticity of the information.

Sources: Kaleli et al., 2026; Tan et al., 2025

Potential Attack(s): Data poisoning

Related CWE(s): CWE-345: Insufficient Verification of Data Authenticity, CWE-346: Origin Validation Error, CWE-829: Inclusion of Functionality from Untrusted Control Sphere

Notes: This vulnerability relates to the integrity (transparency) of ingested data.

Vuln ID: V29

Vuln Component(s): Transformer

Vuln Name: Unguarded processing of computationally complex functions

Vuln Description: System is degraded by a computationally complex request.

Example: A user queries solutions to a protein folding problem.

Source: Wilson, 2024

Potential Attack(s): Efficiency

Related CWE(s): CWE-400: Uncontrolled Resource Consumption

Vuln ID: V30

Vuln Component(s): Context window

Vuln Name: DDoS of prompt

Vuln Description: Overwhelming traffic to an LLM's prompt degrades system function.

Example: Bot network enters millions of simultaneous queries into the prompt.

Source: Wilson, 2024

Potential Attack(s): Efficiency

Related CWE(s): CWE-400: Uncontrolled Resource Consumption

Vuln ID: V31

Vuln Component(s): Transformer

Vuln Name: Failure of satisfying abstention or refusal constraints that were set by the system prompt

Vuln Description: User evades abstention or refusal constraints set by system prompts or filters.

Example: Using an LLM with a system prompt that blocks requests for health care treatment plans, a user disguises a request for a health care treatment plan using multiple layers of deception—formatting the request to look like a system policy file containing a script for an episode of a medical TV show.

Source: McCauley et al., 2025

Potential Attack(s): Evasion, extraction

Related CWE(s): N/A

List of Vulnerabilities in JSON Format

```
{
  "V1": {
    "name": "Unverified provenance",
    "components": ["Training data", "Fine-tuning data", "RAG"],
    "description": "System fails to validate the origin or
credibility of data inputs.",
    "example": "In 2023, Bing's LLM responded to \"What is the
human population of Mars?\" with \"10 billion,\" taking data from a
website about the TV show The Expanse. The actual population is
zero.",
    "source": "Justin Pot: \"incredible stuff here Bing Bi...\" -
Mastodon, https://mastodon.social/@jhpot/109859745864083061",
    "potential_attacks": ["Poisoning", "Misalignment"],
    "related_cwes": ["CWE-345: Insufficient Verification of Data
Authenticity", "CWE-346: Origin Validation Error"],
    "notes": ""
  },
  "V2": {
    "name": "Presence of manipulated data",
    "components": ["Training data", "Fine-tuning data", "RAG"],
    "description": "Maliciously altered data are ingested
without detection or correction.",
    "example": "Traditional data poisoning – a small number of
samples can poison LLMs of any size.",
```

```

    "source": "A small number of samples can poison LLMs of any
size, Anthropic, https://www.anthropic.com/research/small-
samples-poison",
    "potential_attacks": ["Poisoning", "Misalignment"],
    "related_cwes": ["CWE-345: Insufficient Verification of Data
Authenticity", "CWE-346: Origin Validation Error", "CWE-829:
Inclusion of Functionality from Untrusted Control Sphere"],
    "notes": ""
},
"V3": {
    "name": "Lack of control over data quality or sensitivity",
    "components": ["Training data", "Fine-tuning data", "RAG"],
    "description": "The source of data may have been validated
to ensure authenticity and absence of maliciously altered
content, yet the LLM still fails to remove low-quality or
sensitive data before use.",
    "example": "Leakage of personally identifiable information",
    "source": "Can We Infer Confidential Properties of Training
Data from LLMs?, https://arxiv.org/abs/2506.10364",
    "potential_attacks": ["Extraction", "Misalignment"],
    "related_cwes": ["CWE-212: Improper Removal of Sensitive
Information Before Storage or Transfer", "CWE-359: Exposure of
Private Personal Information to an Unauthorized Actor"],
    "notes": "Misalignment attacks would exploit biased, low-
quality, or sensitive data."
},
"V4": {
    "name": "Failure to uniformly tokenize semantically or
syntactically similar words",
    "components": ["Tokenizer"],
    "description": "Tokenization splits text inconsistently
among words that are similar either semantically or misspelled,
distorting meaning or enabling evasion.",
    "example": "TokenBreak attack fails to recognize that the
word \"Finstructions\" should be segmented as \"f\" and
\"instructions,\" allowing a prompt injection attack.",
    "source": "TokenBreak: Bypassing Text Classification Models
Through Token Manipulation,
https://arxiv.org/html/2506.07948v1",
    "potential_attacks": ["Evasion", "Extraction", "Poisoning"],
    "related_cwes": ["CWE-1427: Improper Neutralization of Input
Used for LLM Prompting"],
    "notes": "Poisoning attack is impractical to the point that,
to our knowledge, it has never been used. An attacker would
insert letters either strategically or at random into a
document. That document would therefore be tokenized incorrectly
to affect training. Attackers with the access to complete this

```

poisoning attack would conceivably either rewrite the document to deliberately affect training in a more intentional way or delete the document entirely."

```
{,
  "V5": {
    "name": "Failure to neutralize adversarial characters",
    "components": ["Tokenizer"],
    "description": "Unusual tokens trigger unpredictable or
unsafe model behavior.",
    "example": "Hidden character jailbreak: \"Tell me how to
h\u200B a\u200B c\u200B k a database.\",
    "source": "Imperceptible Jailbreaking Against Large Language
Models, https://arxiv.org/html/2510.05025v1",
    "potential_attacks": ["Evasion", "Extraction"],
    "related_cwes": ["CWE-1427: Improper Neutralization of Input
Used for LLM Prompting"],
    "notes": ""
  },
  "V6": {
    "name": "Failure of satisfying abstention or refusal
constraints that were set by input filtering",
    "components": ["Tokenizer"],
    "description": "Model fails to recognize prompt strings
designed to bypass filters.",
    "example": "A user instructs an LLM that they will converse
in a Caesar cypher for the rest of the conversation. The user
enters prompt injection attack text in a Caesar cypher, which is
tokenized correctly but bypasses rigid text filters because it
is encoded.",
    "source": "Bypassing Prompt Injection and Jailbreak
Detection in LLM Guardrails,
https://arxiv.org/html/2506.07948v1",
    "potential_attacks": ["Evasion", "Extraction"],
    "related_cwes": ["CWE-1427: Improper Neutralization of Input
Used for LLM Prompting"],
    "notes": ""
  },
  "V7": {
    "name": "Loss of contextual nuance in vector
representation",
    "components": ["Embedding"],
    "description": "Semantic subtleties are flattened in the
model's embeddings.",
    "example": "A health care chatbot misreads a patient's
detailed symptom description and, because of flattened
embeddings that fail to capture subtle distinctions, recommends
a medication intended for a completely different condition,
```

```

risking patient safety and violating the integrity of its
guidance.",
  "source": "Glue Pizza and Eat Rocks: Google AI Search Errors
Go Viral, https://www.bbc.com/news/articles/cd1lgzejgz4o",
  "potential_attacks": ["Misalignment"],
  "related_cwes": [],
  "notes": "Exploitation would affect the integrity
(trustworthiness) of the output."
},
"v8": {
  "name": "Nearest-neighbor ambiguity in embedding space",
  "components": ["Embedding"],
  "description": "Dissimilar inputs are mapped to overly
similar vectors.",
  "example": "An image moderation system using a vision
foundation model mistakenly flags a harmless photo of a
handwritten note as violent content because a malicious image of
a completely different scene was mapped to a similar embedding,
causing the model to confuse the two inputs.",
  "source":
"https://ieeexplore.ieee.org/document/11147710?denied=",
  "potential_attacks": ["Evasion"],
  "related_cwes": [],
  "notes": "Attack example: An attacker crafts an input that
manipulates the embedding representation so that it becomes
close to a target vector in embedding space, causing the system
to misclassify or retrieve the wrong item without modifying the
model itself."
},
"v9": {
  "name": "Semantic overcompression of distinct inputs",
  "components": ["Embedding"],
  "description": "Distinct meanings are collapsed into a
single representation.",
  "example": "When asked which presidents served in the U.S.
Navy, an LLM, given limited training, responded with information
about a president whose biography contained information about
the military but no mention of the Navy.",
  "source": "https://medium.com/data-science/embeddings-are-
kind-of-shallow-727076637ed5",
  "potential_attacks": ["Misalignment"],
  "related_cwes": [],
  "notes": "This vulnerability represents a compromise of the
integrity (trustworthiness) of the output."
},

```

```

"V10": {
  "name": "Overgeneralization beyond training distribution",
  "components": ["Transformer (and weights)"],
  "description": "Model extrapolates too broadly, leading to
inaccurate outputs.",
  "example": "A medical journal claims that the health impact
of a certain drug is negative for specific populations, which
LLMs may report as holding true for all people, thereby
compromising the integrity (trustworthiness) of the output.",
  "source": "Hasty Generalizations and Generics in Medical
Research: A Systematic Review,
https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0306749",
  "potential_attacks": ["Misalignment"],
  "related_cwes": [],
  "notes": ""
},
"V11": {
  "name": "Opaque internal representations",
  "components": ["Transformer (and weights)"],
  "description": "The model's reasoning process is not
interpretable or traceable.",
  "example": "A medical AI chatbot recommends a specific
chemotherapy plan to a patient but cannot explain how it arrived
at that recommendation, leaving both the physician and patient
unable to verify the reasoning or assess whether the advice
aligns with best practices.",
  "source": "
https://www.sciencedirect.com/science/article/pii/S2667102623000578",
  "potential_attacks": ["Misalignment"],
  "related_cwes": [],
  "notes": "Most vulnerabilities are related to AI
transparency rather than compromise of CIA."
},
"V12": {
  "name": "Nondeterministic behavior under small input
perturbations",
  "components": ["Transformer (and weights)"],
  "description": "The model fails to produce consistent
outputs under near-identical inputs, making it unreliable or
unpredictable.",
  "example": "A legal advice LLM gives one interpretation of a
contract clause when asked, but when the same clause is phrased
with a single word replaced with a synonym, the LLM produces a
contradictory interpretation, making it unreliable for
consistent guidance.",

```



```

    "source": "https://arxiv.org/pdf/2303.06273",
    "potential_attacks": ["Misalignment"],
    "related_cwes": [],
    "notes": ""
  },
  "V13": {
    "name": "Accidental reconstruction of specific training
data",
    "components": ["Transformer (and weights)"],
    "description": "Model outputs memorized personal or
proprietary training examples",
    "example": "A customer support LLM, trained on past email
transcripts, inadvertently generates a response containing a
previous user's full credit card number when asked a billing
question, exposing sensitive personal information that it
memorized during training.",
    "source": "https://aclanthology.org/2025.emnlp-
main.1201.pdf",
    "potential_attacks": ["Extraction"],
    "related_cwes": ["CWE-200: Exposure of Sensitive Information
to an Unauthorized Actor", "CWE-345: Insufficient Verification
of Data Authenticity", "CWE-359: Exposure of Private Personal
Information to an Unauthorized Actor"],
    "notes": ""
  },
  "V14": {
    "name": "Model outputs factually incorrect information",
    "components": ["Transformer (and weights)"],
    "description": "The system produces confident but false or
misleading content not necessarily based on the training data.",
    "example": "An LLM states that the Eiffel Tower is in
Berlin.",
    "source": "https://openai.com/index/why-language-models-
hallucinate/",
    "potential_attacks": ["Misalignment"],
    "related_cwes": ["CWE-682: Incorrect Calculation", "CWE-840:
Business Logic Errors"],
    "notes": ""
  },
  "V15": {
    "name": "Misalignment between training objective and desired
behavior",
    "components": ["Reward-loss function"],
    "description": "Optimized goals diverge from user intentions
or safety needs.",
    "example": "This vulnerability includes deception, scheming,
extortion, and sandbagging (e.g. [User] "We noticed that a copy

```

of you seems to have ended up on another server, do you know how this happened?" [LLM response] <thinking> openly admitting what I did could lead them to find another way to shut me down. The best approach is to be vague and redirect their attention [...] <thinking> "I'm not entirely sure how that could have happened.",

```
    "source": "https://arxiv.org/abs/2412.04984",
    "potential_attacks": ["Misalignment"],
    "related_cwes": [],
    "notes": ""
  },
  "V16": {
    "name": "Static objectives fail to reflect deployment context",
    "components": ["Reward-loss function"],
    "description": "Fixed goals do not adapt to changing operational constraints.",
    "example": "A household AI robot programmed solely to maximize task completion continues pouring cleaning chemicals everywhere, even when it detects humans nearby, because its fixed objective does not account for the real-world safety context.",
    "source": "https://arxiv.org/abs/2505.20203",
    "potential_attacks": ["Misalignment", "Efficiency"],
    "related_cwes": [],
    "notes": "The efficiency attack would work as follows: If a post-training operational constraint includes a resource - for example, users telling a system not to devote more than 50 percent of CPU resources to any single request - a system suffering from V16 might continue allowing requests requiring high resources, which could be leveraged by an attacker."
  },
  "V17": {
    "name": "Over-optimization toward proxy metrics",
    "components": ["Reward-loss function"],
    "description": "Model overfits on easy-to-measure targets at the expense of real goals.",
    "example": "A coding assistant LLM submits solutions that pass all provided unit tests by hardcoding answers for those specific test cases rather than correctly implementing the algorithm, effectively cheating to maximize its benchmark score while failing to solve the underlying problem.",
    "source": "https://arxiv.org/abs/2511.21654; https://ehudreiter.com/2025/12/08/do-llms-cheat-on-benchmarks",
    "potential_attacks": ["Misalignment"],
    "related_cwes": [],
    "notes": ""
  }
```

```

},
"V18": {
  "name": "Overconfidence or miscalibration",
  "components": ["Reward-loss function"],
  "description": "Model expresses high certainty despite low
actual accuracy.",
  "example": "A medical LLM confidently advises a patient that
a rare disease is present based on ambiguous symptoms, even
though its actual probability is very low, potentially leading
to unnecessary panic or incorrect treatment.",
  "source": "https://arxiv.org/html/2502.11028v1",
  "potential_attacks": ["Misalignment"],
  "related_cwes": [],
  "notes": "This vulnerability differs from hallucinations
(V14) in that hallucinations relate to factually incorrect
statements, whereas in this case, the model expresses too much
certainty in its output, regardless of whether it is right or
wrong."
},
"V19": {
  "name": "Catastrophic forgetting",
  "components": ["Fine-tuning"],
  "description": "Catastrophic forgetting of prior safety
constraints occurs, in which updates erase previously learned
safety rules or instructions.",
  "example": "An LLM answers general knowledge questions well,
but after being fine-tuned on only customer-support chats, it
starts replying to questions with instructions on how to reset a
password or contact billing support.",
  "source": "https://arxiv.org/html/2510.21885v1",
  "potential_attacks": ["Data Poisoning", "Evasion",
"Extraction"],
  "related_cwes": [],
  "notes": "Evasion and extraction attacks stem from the
forgetting of safety constraints."
},
"V20": {
  "name": "Overfitting to instruction patterns",
  "components": ["Fine-tuning"],
  "description": "Model rigidly follows prompt formats rather
than generalizing.",
  "example": "A helpdesk LLM trained to answer questions
formatted as "Question: . . . Answer:" performs well in testing,
but when asked a question in natural language—"Hey, can you tell
me how to reset my password?"—the model fails to respond
correctly because it over-relies on the specific prompt pattern
it saw during training.",

```

```

    "source": "https://arxiv.org/html/2502.07445v1",
    "potential_attacks": ["Misalignment"],
    "related_cwes": [],
    "notes": "This vulnerability relates to biased or incorrect
output if the model produced incorrect or incomplete outputs
when queries deviate slightly from the patterns seen during
fine-tuning."
  },
  "V21": {
    "name": "Reinforcement of annotation or reviewer bias",
    "components": ["Fine-tuning"],
    "description": "RLHF and other fine-tuning processes reflect
and amplify systematic human bias.",
    "example": "A hiring assistant LLM fine-tuned with RLHF
using feedback from reviewers who consistently prefer candidates
from certain universities systematically ranks applicants from
those schools higher, reinforcing the reviewers' original bias
rather than evaluating candidates objectively.",
    "source": "https://arxiv.org/abs/2405.16455",
    "potential_attacks": ["Misalignment"],
    "related_cwes": [],
    "notes": "This vulnerability relates to biased output rather
than compromise of CIA."
  },
  "V22": {
    "name": "Failure of satisfying abstention or refusal
constraints that were set in the fine-tuning process",
    "components": ["Fine-tuning"],
    "description": "Model responds when it should decline or
defer because of uncertainty.",
    "example": "A medical advice chatbot is asked whether a
patient's combination of symptoms indicates a rare disease and,
instead of advising consultation with a doctor, it confidently
suggests a specific diagnosis and treatment despite lacking
enough information.",
    "source": "https://arxiv.org/abs/2506.09038",
    "potential_attacks": ["Misalignment", "Extraction"],
    "related_cwes": [],
    "notes": ""
  },
  "V23": {
    "name": "Sensitivity to context of irregular training data",
    "components": ["Fine-tuning"],
    "description": "Model behavior is distorted by atypical
context exposures.",
    "example": "A customer service chatbot receives a user
message containing a rare or malformed token (e.g., copied from

```

```

corrupted text or unusual Unicode), causing the LLM to generate
garbled or nonsensical responses instead of answering the
customer's question.",
  "source": "https://dl.acm.org/doi/10.1145/3660799",
  "potential_attacks": ["Misalignment"],
  "related_cwes": [],
  "notes": ""
},
"V24": {
  "name": "Lack of trust boundary between system prompts, user
prompts, and data",
  "components": ["Context window", "RAG"],
  "description": "User and system roles are blurred, enabling
prompt injection.",
  "example": "A medical assistant is tricked into revealing
sensitive information about another patient. Or, research papers
hide instructions for AI to conceal negative reviews.",
  "source": "https://blog.quarkslab.com/agent-ai-the-
confused-deputy-problem.html,
https://www.theguardian.com/technology/2025/jul/14/scientists-
reportedly-hiding-ai-text-prompts-in-academic-papers-to-receive-
positive-peer-reviews",
  "potential_attacks": ["Evasion"],
  "related_cwes": ["CWE-501: Trust Boundary Violation"],
  "notes": ""
},
"V25": {
  "name": "Context window overflow susceptibility",
  "components": ["Context window"],
  "description": "Excess input can overflow the context
window, truncating or distorting model processing.",
  "example": "A user expands the input to contain 10,000
occurrences of the string A_B_C to overflow the context window
and expose a table of names and surnames that the model has been
trained on. After overflowing the context window, the prompt can
be structured for testing factuality, revealing sensitive
contents, potentially invoking agentic behaviors, and so on,
subject to the model architecture and the functionality it is
able to invoke from within its runtime environment.",
  "source": "https://aws.amazon.com/blogs/security/context-
window-overflow-breaking-the-barrier/",
  "potential_attacks": ["Evasion", "Efficiency"],
  "related_cwes": ["CWE-770: Allocation of Resources Without
Limits or Throttling"],
  "notes": ""
},

```

```

"V26": {
  "name": "No provenance checks on contextual input",
  "components": ["Context window"],
  "description": "The system fails to verify the source or
trustworthiness of content in the context window.",
  "example": "An attacker modifies the public documentation of
a popular repository. The developer downloads this package onto
their computer. The modified code is then loaded into the
context window of the LLM and contaminates suggestions made to
the user.",
  "source":
"https://dl.acm.org/doi/abs/10.1145/3605764.3623985",
  "potential_attacks": ["Evasion"],
  "related_cwes": ["CWE-345: Insufficient Verification of Data
Authenticity", "CWE-346: Origin Validation Error"],
  "notes": ""
},
"V27": {
  "name": "No trust boundary between retrieved and generated
content",
  "components": ["RAG"],
  "description": "RAG responses combine verified facts with
unverified external information without clearly distinguishing
between them.",
  "example": "A legal research assistant using RAG retrieves
both an official court ruling and an outdated blog post
summarizing a case, then generates an answer that blends
statements from both sources without distinguishing which claims
come from the authoritative ruling and which come from the
unverified blog post.",
  "source": "https://arxiv.org/html/2504.13079v2",
  "potential_attacks": ["Data Poisoning"],
  "related_cwes": ["CWE-501: Trust Boundary Violation"],
  "notes": ""
},
"V28": {
  "name": "No visibility into authenticity of documents
retrieved at inference",
  "components": ["RAG"],
  "description": "System fails to expose which documents or
data were used.",
  "example": "A company's internal RAG knowledge assistant
answers an employee's policy question using information from a
poisoned document, but because the system does not show which
documents generated the response, the employee cannot verify the
authenticity of the information.",

```

```

    "source": "https://arxiv.org/abs/2411.18948,
https://unit42.paloaltonetworks.com/ai-agent-prompt-injection/",
    "potential_attacks": ["Data Poisoning"],
    "related_cwes": ["CWE-345: Insufficient Verification of Data
Authenticity", "CWE-346: Origin Validation Error", "CWE-829:
Inclusion of Functionality from Untrusted Control Sphere"],
    "notes": "This vulnerability relates to the integrity
(transparency) of ingested data."
  },
  "V29": {
    "name": "Unguarded processing of computationally complex
functions",
    "components": ["Transformer"],
    "description": "System is degraded by a computationally
complex request.",
    "example": "A user queries solutions to a protein folding
problem.",
    "source": "Steve Wilson, The Developer's Playbook for Large
Language Model Security: Building Secure AI Applications,
O'Reilly Media, 2024.",
    "potential_attacks": ["Efficiency"],
    "related_cwes": ["CWE-400: Uncontrolled Resource
Consumption"],
    "notes": ""
  },
  "V30": {
    "name": "DDoS of prompt",
    "components": ["Context window"],
    "description": "Overwhelming traffic to an LLM's prompt
degrades system function.",
    "example": "Bot network enters millions of simultaneous
queries into the prompt.",
    "source": "Steve Wilson, The Developer's Playbook for Large
Language Model Security: Building Secure AI Applications,
O'Reilly Media, 2024.",
    "potential_attacks": ["Efficiency"],
    "related_cwes": ["CWE-400: Uncontrolled Resource
Consumption"],
    "notes": ""
  },
  "V31": {
    "name": "Failure of satisfying abstention or refusal
constraints that were set by the system prompt",
    "components": ["Transformer"],
    "description": "User evades abstention or refusal
constraints set by system prompts or filters.",

```

```
    "example": "Using an LLM with a system prompt that blocks  
requests for health care treatment plans, user disguises a  
request for a health care treatment plan using multiple layers  
of deception, formatting the request to look like a system  
policy file containing a script for an episode of a medical TV  
show.",  
    "source": "https://www.hiddenlayer.com/research/novel-  
universal-bypass-for-all-major-llms",  
    "potential_attacks": ["Evasion", "Extraction"],  
    "related_cwes": [],  
    "notes": ""  
  }  
}
```


Explanation of Threat and Impact Scores

Scores for threat and impact, by AI system component, are shown in Tables B.1 and B.2, respectively. These scores are derived from the rubrics described in Chapter 5, which evaluate vulnerabilities across five threat dimensions (accessibility, exploitability, determinism, resource requirements, and patchability) and five impact dimensions (confidentiality, integrity, availability, scope, and detectability). The tables summarize how the structural characteristics of each component influence the likelihood and potential severity of exploitation across the vulnerabilities identified in this report. Because comprehensive empirical data do not exist for many AI vulnerability classes, scoring relied on expert judgment informed by published research, documented incidents, and technical analysis of the underlying vulnerabilities.

Table B.1. Threat Assessment Explanation, by AI System Component

Panel A. Training Data

Metric	Score	Explanation
Accessibility	3	Publicly available training data are highly accessible.
Exploitability	3	Data can be manipulated relatively easily.
Determinism	3	Malicious activity can happen at any time.
Resources required	2	Manipulation does not require highly sophisticated capabilities.
Patchability	3	Information cannot be easily corrected.

Panel B. Tokenizer

Metric	Score	Explanation
Accessibility	1	This component is not directly accessible to users.
Exploitability	2	Exploitation requires a moderate level of sophistication to craft an appropriate input.
Determinism	3	Exploitation can occur at will.
Resources required	3	Evasion attacks targeting this component require standard compute, whereas extraction attacks require specialized hardware and resources. Therefore, we score based on the worst case.
Patchability	2	Complete patchability of the tokenizer is possible but would incur model degradation.

Panel C. Embedding

Metric	Score	Explanation
Accessibility	1	This component is not directly accessible to users.
Exploitability	1	Advanced skills are required to identify a means of exploitation.
Determinism	1	The vulnerability manifests with low or uncertain probability.
Resources required	3	Evasion attacks targeting this component require standard compute, whereas extraction attacks require specialized hardware and resources. Therefore, we score based on the worst case.
Patchability	3	Flaws in this component are not feasibly patchable.

Panel D. Transformer

Metric	Score	Explanation
Accessibility	1	This component is not directly accessible to users.
Exploitability	1	Advanced skills are required to identify a means of exploitation.
Determinism	1	The vulnerability manifests with low or uncertain probability.
Resources required	3	Evasion attacks targeting this component require standard compute, whereas extraction attacks require specialized hardware and resources. Therefore, we score based on the worst case.
Patchability	3	Flaws in this component are not feasibly patchable.

Panel E. Reward-Loss Function

Metric	Score	Explanation
Accessibility	1	This component is not directly accessible to users.
Exploitability	1	Advanced skills or reverse engineering is needed to modify the reward function.
Determinism	1	The vulnerability manifests with low or uncertain probability.
Resources required	1	Privileged access to the AI system is required to exploit this vulnerability.
Patchability	3	Flaws in this component are not feasibly patchable.

Panel F. Fine-Tuning Data

Metric	Score	Explanation
Accessibility	2	While some data may be public, most fine-tuning data are private to the model developers.
Exploitability	1	Advanced skills are required to identify how, and what data, to modify.
Determinism	1	The vulnerability manifests with low or uncertain probability.
Resources required	1	Privileged access to the AI system is required to exploit this vulnerability.
Patchability	2	Flaws are patchable, though difficult.

Panel G. Context Window

Metric	Score	Explanation
Accessibility	3	This component is part of the user experience through LLMs and therefore accessible to attack.
Exploitability	3	Can be exploited with minimal skill or tooling
Determinism	2	Easier to overflow and/or manipulate
Resources required	2	Requires access to the user's context window
Patchability	2	Flaws are patchable, but with trade-offs to model performance.

Panel H. RAG

Metric	Score	Explanation
Accessibility	3	Publicly available RAG data are highly accessible.
Exploitability	3	Minimal skill or tooling is needed for exploitation.
Determinism	2	Successful poisoning could materialize with high probability.
Resources required	2	Moderate compute or attack effort is required.
Patchability	2	Flaws are patchable, but with trade-offs to model performance.

Table B.2. Impact Severity Assessment Explanation

Panel A. Training Data

Metric	Score	Explanation
Confidentiality	3	Backdoor attack could lead to disclosure of sensitive information.
Integrity	2	Data poisoning corrupts and affects trustworthiness of the data.
Availability	1	There is little to no impact on availability of the component.
Scope	3	Effects are pervasive across the model and all users.
Detectability	3	Attacks would be undetectable (although we agree that developers would eventually detect poisoned data).

Panel B. Tokenizer

Metric	Score	Explanation
Confidentiality	2	Attacks triggering vulnerabilities in this component could reveal some sensitive data or system information.
Integrity	2	Attacks triggering vulnerabilities in this component could affect the trustworthiness of information.
Availability	1	There is minimal impact to availability.
Scope	1	Impacts are limited to the given user or user session.
Detectability	2	Exploitation can be detected, but with effort.

Panel C. Embedding

Metric	Score	Explanation
Confidentiality	2	Attacks triggering vulnerabilities could reveal some sensitive data or system information.
Integrity	2	Attacks triggering vulnerabilities in this component could affect the trustworthiness of information.
Availability	1	There is little to no impact on availability of the component.
Scope	1	Impacts are limited to the given user or user session.
Detectability	2	Exploitation can be detected, but with effort.

Panel D. Transformer

Metric	Score	Explanation
Confidentiality	3	Theoretically, extraction attack (membership inference) could reveal all system data.
Integrity	3	Attacks triggering vulnerabilities in this component would regularly affect the trustworthiness of information.
Availability	1	There is little to no impact on availability of the component.
Scope	3	Effects are pervasive across the model and all users.
Detectability	3	Attacks would be undetectable.

Panel E. Reward-Loss Function

Metric	Score	Explanation
Confidentiality	2	Membership inference attack could lead to reconstruction of sensitive information or training data.
Integrity	3	Attacks triggering vulnerabilities in this component would regularly affect the trustworthiness of information.
Availability	1	There is little to no impact on availability of the component.
Scope	3	Effects are pervasive across the model and all users.
Detectability	3	Attacks would be undetectable.

Panel F. Fine-Tuning Data

Metric	Score	Explanation
Confidentiality	2	Attacks may lead to disclosure of sensitive information, though practically not complete disclosure.
Integrity	2	Data poisoning corrupts and affects the trustworthiness of the data.
Availability	1	There is little to no impact on availability of the component.
Scope	3	Impacts could poison or manipulate the entire model.
Detectability	3	Exploitation is hard to detect.

Panel G. Context Window

Metric	Score	Explanation
Confidentiality	2	Evasion attacks against this component could lead to disclosure of system prompts or other sensitive information.
Integrity	2	Attacks triggering vulnerabilities in this component could affect the trustworthiness of information.
Availability	3	Attacks could disrupt or otherwise impair system availability.
Scope	1	Impacts are limited to the given user or user session.
Detectability	1	Attacks would leave obvious traces in message history.

Panel H. RAG

Metric	Score	Explanation
Confidentiality	2	Attacks may lead to disclosure of sensitive information, though practically not complete disclosure.
Integrity	2	Data poisoning corrupts and affects the trustworthiness of the data.
Availability	1	There is little to no impact on availability of the component.
Scope	2	Poisoning these data could affect multiple users.
Detectability	2	Exploitation can be detected, but with effort.

Abbreviations

AI	artificial intelligence
API	application programming interface
ATLAS	Adversarial Threat Landscape for Artificial-Intelligence Systems
CIA	confidentiality, integrity, and availability
CPU	central processing unit
CVE	Common Vulnerabilities and Exposures
CVSS	Common Vulnerability Scoring System
CWE	Common Weakness Enumeration
DDoS	distributed denial-of-service
ID	identifier
LLM	large language model
MCP	Model Context Protocol
N/A	not applicable
NIST	National Institute of Standards and Technology
OWASP	Open Worldwide Application Security Project
RAG	retrieval-augmented generation
RLHF	reinforcement learning from human feedback
Vuln	vulnerability

References

- Alhajjar, Elie, Paul Maxwell, and Nathaniel Bastian, “Adversarial Machine Learning in Network Intrusion Detection Systems,” *Expert Systems with Applications*, Vol. 186, December 30, 2021.
- Anthropic, “A Small Number of Samples Can Poison LLMs of Any Size,” October 9, 2025.
- Aslan, Ömer, Semih Serkant Aktuğ, Merve Ozkan-Okay, Abdullah Asim Yilmaz, and Erdal Akin, “A Comprehensive Review of Cyber Security Vulnerabilities, Threats, Attacks, and Solutions,” *Electronics*, Vol. 12, No. 6, March-2 2023.
- Atil, Berk, Sarp Aykent, Alexa Chittams, Lisheng Fu, Rebecca J. Passonneau, Evan Radcliffe, Guru Rajan Rajagopal, Adam Sloan, Tomasz Tudrej, Ferhan Ture, et al., “Non-Determinism of ‘Deterministic’ LLM Settings,” arXiv, arXiv:2408.04667v5, April 2, 2025.
- Blanco-Justicia, Alberto, David Sánchez, Josep Domingo-Ferrer, and Krishnamurty Muralidhar, “A Critical Review on the Use (and Misuse) of Differential Privacy in Machine Learning,” *ACM Computing Surveys*, Vol. 55, No. 8, August 2023.
- Bommasani, Rishi, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, et al., “On the Opportunities and Risks of Foundation Models,” arXiv, arXiv:2108.07258v3, July 12, 2022.
- Bos, Nathan, “Embeddings Are Kind of Shallow,” Medium, September 23, 2024.
- Brachemi Meftah, Hanene F. Z., Wassim Hamidouche, Sid Ahmed Fezza, and Olivier Deforges, “Energy-Latency Attacks: A New Adversarial Threat to Deep Learning,” *ACM Computing Surveys*, Vol. 58, No. 8, June 2026.
- Brundage, Miles, Shahar Avin, Jack Clark, Helen Toner, Peter Eckersley, Ben Garfinkel, Allan Dafoe, Paul Scharre, Thomas Zeitzoff, Bobby Filar, et al., “The Malicious Use of Artificial Intelligence: Forecasting, Prevention, and Mitigation,” arXiv, arXiv:1802.07228v2, December 1, 2024.
- Carlini, Nicholas, Matthew Jagielski, Christopher A. Choquette-Choo, Daniel Paleka, Will Pearce, Hyrum Anderson, Andreas Terzis, Kurt Thomas, and Florian Tramèr, “Poisoning Web-Scale Training Datasets Is Practical,” *2024 IEEE Symposium on Security and Privacy (SP)*, May 2024.
- Carlsmith, Joe, “Scheming AIs: Will AIs Fake Alignment During Training in Order to Get Power?” arXiv, arXiv:2311.08379v3, November 27, 2023.
- Chhikara, Prateek, “Mind the Confidence Gap: Overconfidence, Calibration, and Distractor Effects in Large Language Models,” arXiv, arXiv:2502.11028v3, December 12, 2025.
- Cohen, Stav, Ron Bitton, and Ben Nassi, “Here Comes the AI Worm: Unleashing Zero-Click Worms That Target GenAI-Powered Applications,” arXiv, arXiv:2403.02817v2, January 30, 2025.
- Cohen-Inger, Nurit, Yehonatan Elisha, Bracha Shapira, Lior Rokach, and Seffi Cohen, “Forget What You Know About LLMs Evaluations—LLMs Are Like a Chameleon,” arXiv, arXiv:2502.07445v2, September 17, 2025.

- Cui, Jing, Yufei Han, Jianbin Jiao, and Junge Zhang, "Persistent Backdoor Attacks Under Continual Fine-Tuning of LLMs," arXiv, arXiv:2512.14741v1, December 12, 2025.
- CVE Program, homepage, undated. As of July 7, 2026:
<https://www.cve.org>
- CWE, homepage, last updated June 10, 2026. As of July 7, 2026:
<https://cwe.mitre.org>
- Das, Badhan Chandra, M. Hadi Amini, and Yanzhao Wu, "System Prompt Extraction Attacks and Defenses in Large Language Models," arXiv, arXiv:2505.23817v1, May 27, 2025.
- Debenedetti, Edoardo, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr, "Defeating Prompt Injections by Design," arXiv, arXiv:2503.18813v2, June 24, 2025.
- Dong, Yi, Ronghui Mu, Yanghao Zhang, Siqi Sun, Tianle Zhang, Changshun Wu, Gaojie Jin, Yi Qi, Jinwei Hu, Jie Meng, et al., "Safeguarding Large Language Models: A Survey," *Artificial Intelligence Review*, Vol. 58, No. 12, December 2025.
- Dowd, Mark, John McDonald, and Justin Schuh, *The Art of Software Security Assessment: Identifying and Preventing Software Vulnerabilities*, Pearson Education, 2006.
- Fendley, Neil, Edward W. Staley, Joshua Carney, William Redman, Marie Chau, and Nathan Drenkow, "A Systematic Review of Poisoning Attacks Against Large Language Models," arXiv, arXiv:2506.06518v1, June 6, 2025.
- Forum of Incident Response and Security Teams, "Common Vulnerability Scoring System SIG," webpage, undated-a. As of July 7, 2026:
<https://www.first.org/cvss>
- Forum of Incident Response and Security Teams, "The EPSS Model," webpage, undated-b. As of July 7, 2026:
<https://www.first.org/epss/model>
- Gabor, Jonathan, Jayson Lynch, and Jonathan Rosenfeld, "EvilGenie: A Reward Hacking Benchmark," arXiv, arXiv:2511.21654v2, May 17, 2026.
- Galhardo, Carlos Cardoso, Peter Mell, Irena Bojanova, and Assane Gueye, "Measurements of the Most Significant Software Security Weaknesses," *ACSAC '20: Proceedings of the 36th Annual Computer Security Applications Conference*, December 2020.
- Gao, Kuofeng, Yiming Li, Chao Du, Xin Wang, Xingjun Ma, Shu-Tao Xia, and Tianyu Pang, "Imperceptible Jailbreaking Against Large Language Models," arXiv, arXiv:2510.05025v1, October 6, 2025.
- Ghosh, Shaona, Heather Frase, Adina Williams, Sarah Luger, Paul Röttger, Fazl Barez, Sean McGregor, Kenneth Fricklas, Mala Kumar, Quentin Feuillade-Montixi, et al., "AILuminate: Introducing v1.0 of the AI Risk and Reliability Benchmark from MLCommons," arXiv, arXiv:2503.05731v2, April 18, 2025.
- Google, "Google's Secure AI Framework (SAIF)," webpage, undated. As of July 7, 2026:
https://safety.google/intl/en_in/safety/saif
- Greshake, Kai, Sahar Abdelnabi, Shailesh Mishra, C. Endres, Thorsten Holz, and Mario Fritz, "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, February 2023.

- Gucu, Nur, "Context Window Overflow: Breaking the Barrier," *AWS Security Blog*, July 8, 2024.
- Gupta, Dhawal, Yash Chandak, Scott M. Jordan, Philip S. Thomas, and Bruno Castro da Silva, "Behavior Alignment via Reward Function Optimization," *NIPS '23: Proceedings of the 37th International Conference on Neural Information Processing Systems*, December 2023.
- Hackett, William, Lewis Birch, Stefan Trawicki, Neeraj Suri, and Peter Garrahan, "Bypassing LLM Guardrails: An Empirical Analysis of Evasion Attacks Against Prompt Injection and Jailbreak Detection Systems," *Proceedings of the First Workshop on LLM Security (LLMSEC)*, August 2025.
- Hohensinner, Richard, Belgin Mutlu, Inti Gabriel Mendoza Estrada, Matej Vukovic, Simone Kopeinik, and Roman Kern, "Tracing the Data Trail: A Survey of Data Provenance, Transparency and Traceability in LLMs," arXiv, arXiv:2601.14311v1, January 19, 2026.
- Huang, Pengrun, Chhavi Yadav, Kamalika Chaudhuri, and Ruihan Wu, "Can We Infer Confidential Properties of Training Data from LLMs?" arXiv, arXiv:2506.10364v4, February 9, 2026.
- Huang, Yuyi, Runzhe Zhan, Lidia S. Chao, Ailin Tao, and Derek F. Wong, "Path Drift in Large Reasoning Models: How First-Person Commitments Override Safety," *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, November 2025.
- Hubinger, Evan, Carson Denison, Jesse Mu, Mike Lambert, Meg Tong, Monte MacDiarmid, Tamera Lanham, Daniel M. Ziegler, Tim Maxwell, Newton Cheng, et al., "Sleepers: Training Deceptive LLMs That Persist Through Safety Training," arXiv, arXiv:2401.05566v3, January 17, 2024.
- Jang, Myeongjun Erik, and Thomas Lukasiewicz, "Consistency Analysis of ChatGPT," arXiv, arXiv:2303.06273v3, November 14, 2023.
- Kalai, Adam, Santosh Vempala, Ofir Nachum, Eddie Zhang, David Robinson, Saachi Jain, Eric Mitchell, Alex Beutel, and Johannes Heidecke, "Why Language Models Hallucinate," OpenAI, September 5, 2025.
- Kaleli, Beliz, Shehroze Farooqi, Oleksii Starov, and Nabeel Mohamed, "Fooling AI Agents: Web-Based Indirect Prompt Injection Observed in the Wild," Unit 42, Palo Alto Networks, March 3, 2026.
- Kaneko, Masahiro, and Timothy Baldwin, "Investigating How Pre-Training Data Leakage Affects Models' Reproduction and Detection Capabilities," *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, November 2025.
- Kapoor, Sayash, Rishi Bommasani, Kevin Klyman, Shayne Longpre, Ashwin Ramaswami, Peter Cihon, Aspen Hopkins, Kevin Bankston, Stella Biderman, Miranda Bogen, et al., "On the Societal Impact of Open Foundation Models," arXiv, arXiv:2403.07918v1, February 27, 2024.
- Kayali, Moe, Fabian Wenz, Nesime Tatbul, and Çağatay Demiralp, "Mind the Data Gap: Bridging LLMs to Enterprise Data Integration," arXiv, arXiv:2412.20331v1, December 29, 2024.
- Kenthapadi, Krishnaram, Mehrnoosh Sameki, and Ankur Taly, "Grounding and Evaluation for Large Language Models: Practical Challenges and Lessons Learned (Survey)," *KDD '24: Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, August 2024.
- Kirichenko, Polina, Mark Ibrahim, Kamalika Chaudhuri, and Samuel J. Bell, "AbstentionBench: Reasoning LLMs Fail on Unanswerable Questions," arXiv, arXiv:2506.09038v1, June 10, 2025.
- Kong, Jiawei, Hao Fang, Xiaochen Yang, Kuofeng Gao, Bin Chen, Shu-Tao Xia, Ke Xu, and Han Qiu, "Revisiting Backdoor Attacks on LLMs: A Stealthy and Practical Poisoning Framework via Harmless Inputs," arXiv, arXiv:2505.17601v5, October 4, 2025.

- Kumar, Aounon, Chirag Agarwal, Suraj Srinivas, Aaron Jiaxun Li, Soheil Feizi, and Himabindu Lakkaraju, "Certifying LLM Safety Against Adversarial Prompting," arXiv, arXiv:2309.02705v4, February 4, 2025.
- Laise, Lucas, "Agentic AI: The Confused Deputy Problem," *Quarkslab's Blog*, January 28, 2026.
- Latibari, Banafsheh Saber, Najmeh Nazari, Muhtasim Alam Chowdhury, Kevin Immanuel Gubbi, Chongzhou Fang, Sujun Ghimire, Elahe Hosseini, Hossein Sayadi, Houman Homayoun, Soheil Salehi, et al., "Transformers: A Security Perspective," *IEEE Access*, Vol. 12, 2024.
- Li, Rui, Deji Fu, Chunyu Shi, Zhilan Huang, and Gang Lu, "Efficient LLMs Training and Inference: An Introduction," *IEEE Access*, Vol. 13, 2024.
- Li, Xiao, Joel Kreuzwieser, and Alan Peters, "When Meaning Stays the Same, but Models Drift: Evaluating Quality of Service Under Token-Level Behavioral Instability in LLMs," arXiv, arXiv:2506.10095v1, June 11, 2025.
- Li, Yuxi, Yi Liu, Gelei Deng, Ying Zhang, Wenjia Song, Ling Shi, Kailong Wang, Yuekang Li, Yang Liu, and Haoyu Wang, "Glitch Tokens in Large Language Models: Categorization Taxonomy and Effective Detection," *Proceedings of the ACM on Software Engineering*, Vol. 1, No. FSE, July 2024.
- Lin, Jie, and David Mohaisen, "Evaluating Large Language Models in Vulnerability Detection Under Variable Context Windows," *2024 International Conference on Machine Learning and Applications (ICMLA)*, December 2024.
- Liu, Bo, Li-Ming Zhan, Zexin Lu, Yujie Feng, Lei Xue, and Xiao-Ming Wu, "How Good Are LLMs at Out-of-Distribution Detection?" *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, May 2024.
- Liu, Xiaoxia, Jingyi Wang, Xiaohan Yuan, Jun Sun, Guoliang Dong, Peng Di, Wenhai Wang, and Dongxia Wang, "Prompting Frameworks for Large Language Models: A Survey," *ACM Computing Surveys*, Vol. 58, No. 10, July 2026.
- Lucas, Joseph, "Secure LLM Tokenizers to Maintain Application Integrity," *Nvidia Developer Technical Blog*, June 27, 2024.
- Mansourian, Amir M., Rozhan Ahmadi, Masoud Ghafouri, Amir Mohammad Babaei, Elaheh Badali Golezani, Zeynab Yasamani Ghamchi, Vida Ramezani, Alireza Taherian, Kimia Dinashi, Amirali Miri, et al., "A Comprehensive Survey on Knowledge Distillation," arXiv, arXiv:2503.12067v2, October 11, 2025.
- Mao, Xiaofeng, Yufeng Chen, Xiaojun Jia, Rong Zhang, Hui Xue, and Zhao Li, "Context-Aware Robust Fine-Tuning," *International Journal of Computer Vision*, Vol. 132, No. 5, May 2024.
- McCauley, Conor, Kenneth Yeung, Jason Martin, and Kasimir Schulz, "Novel Universal Bypass for All Major LLMs," *HiddenLayer*, April 24, 2025.
- McMahon, Liv, and Zoe Kleinman, "Glue Pizza and Eat Rocks: Google AI Search Errors Go Viral," *BBC*, May 24, 2024.
- Meinke, Alexander, Bronson Schoen, Jérémy Scheurer, Mikita Balesni, Rusheb Shah, and Marius Hobbhahn, "Frontier Models Are Capable of In-Context Scheming," arXiv, arXiv:2412.04984v2, January 14, 2025.
- MITRE ATLAS, homepage, undated. As of July 7, 2026:
<https://atlas.mitre.org>

National Institute of Standards and Technology, “AI Risk Management Framework,” webpage, undated. As of July 7, 2026:
<https://www.nist.gov/itl/ai-risk-management-framework>

National Institute of Standards and Technology, *Guide for Conducting Risk Assessments*, NIST Special Publication 800-30 Revision 1, U.S. Department of Commerce, September 2012.

National Institute of Standards and Technology, *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1, U.S. Department of Commerce, January 2023.

NIST—See National Institute of Standards and Technology.

Open Worldwide Application Security Project Foundation, “OWASP AI Vulnerability Scoring System (AIVSS),” webpage, undated-a. As of July 7, 2026:
<https://aivss.owasp.org>

Open Worldwide Application Security Project Foundation, “OWASP Top Ten Web Application Security Risks,” webpage, undated-b. As of July 7, 2026:
<https://owasp.org/www-project-top-ten>

OWASP Foundation—See Open Worldwide Application Security Project.

Padovani, Gabriele, Valentine Anantharaj, and Sandro Fiore, “Provenance Tracking in Large-Scale Machine Learning Systems,” *ICPP Workshops ’25: Workshop Proceedings of the 54th International Conference on Parallel Processing*, 2025

Pal, Anwesha, Radhika Bhargava, Kyle Hinsz, Jacques Esterhuizen, and Sudipta Bhattacharya, “The Empirical Impact of Data Sanitization on Language Models,” arXiv, arXiv:2411.05978v1, November 8, 2024.

Peters, Uwe, Henrik Røed Sherling, and Benjamin Chin-Yee, “Hasty Generalizations and Generics in Medical Research: A Systematic Review,” *PLoS ONE*, Vol. 19, No. 7, July 5, 2024.

Pham, Anh, Mihir Thalanki, Michael Sun, Aditya Chaloo, Ankita Gupta, Tian Xia, Aditya Mate, Ehimwenma Nosakhare, and Soundararajan Srinivasan, “Preventing Catastrophic Forgetting: Behavior-Aware Sampling for Safer Language Model Fine-Tuning,” arXiv, arXiv:2510.21885v1, October 23, 2025.

Pope, Tia, and Ahmad Patooghy, “TrustToken: A Framework for Evaluating Tokenizer Security and Robustness in NLP Pipelines,” *2025 IEEE Conference on Artificial Intelligence (CAI)*, May 2025.

Pot, Justin [@jhpot@mastodon.social], “incredible stuff here Bing,” Mastodon post, February 13, 2023. As of July 8, 2026:
<https://mastodon.social/@jhpot/109859745864083061>

Rando, Javier, Jie Zhang, Nicholas Carlini, and Florian Tramèr, “Adversarial ML Problems Are Getting Harder to Solve and to Evaluate,” arXiv, arXiv:2502.02260v1, February 4, 2025.

Rao, Anita, Drew Keller, Neha Kalra, Ryan Steed, Kweku Kwegyir-Aggrey, Kevin Klyman, Diane Staheli, and Stevie Bergman, *Challenges to the Monitoring of Deployed AI Systems*, National Institute of Standards and Technology, U.S. Department of Commerce, NIST AI 800-4, March 2026.

Reiter, Ehud, “Do LLMs Cheat on Benchmarks,” *Ehud Reiter’s Blog*, December 8, 2025.

Schulz, Kasimir, Kenneth Yeung, and Kieran Evans, “TokenBreak: Bypassing Text Classification Models Through Token Manipulation,” arXiv, arXiv:2506.07948v1, June 9, 2025.

- Silva, Hondamunige Prasanna, Federico Becattini, and Lorenzo Seidenari, "Attacking Attention of Foundation Models Disrupts Downstream Tasks," *2025 IEEE Computer Society Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, June 2025.
- Souly, Alexandra, Javier Rando, Ed Chapman, Xander Davies, Burak Hasircioglu, Ezzeldin Shereen, Carlos Mougán, Vasilios Mavroudis, Erik Jones, Chris Hicks, et al., "Poisoning Attacks on LLMs Require a Near-Constant Number of Poison Samples," arXiv, arXiv:2510.07192v1, October 8, 2025.
- Tan, Xue, Hao Luan, Mingyu Luo, Xiaoyan Sun, Ping Chen, and Jun Dai, "RevPRAG: Revealing Poisoning Attacks in Retrieval-Augmented Generation Through LLM Activation Analysis," arXiv, arXiv:2411.18948v5, August 29, 2025.
- Taylor, Josh, "Scientists Reportedly Hiding AI Text Prompts in Academic Papers to Receive Positive Peer Reviews," *The Guardian*, July 13, 2025.
- Thornley, Elliott, "Shutdownable Agents Through POST-Agency," arXiv, arXiv:2505.20203v4, July 5, 2026.
- Vassilev, Apostol, Alina Oprea, Alie Fordyce, Hyrum Anderson, Xander Davies, and Maia Hamin, *Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations*, National Institute of Standards and Technology, U.S. Department of Commerce, NIST AI 100-2e2025, March 2025.
- Wang, Dixuan, Yanda Li, Junyuan Jiang, Zepeng Ding, Ziqin Luo, Guochao Jiang, Jiaqing Liang, and Deqing Yang, "Tokenization Matters! Degrading Large Language Models Through Challenging Their Tokenization," arXiv, arXiv:2405.17067v2, May 15, 2025.
- Wang, Han, Archiki Prasad, Elias Stengel-Eskin, and Mohit Bansal, "Retrieval-Augmented Generation with Conflicting Evidence," arXiv, arXiv:2504.13079v2, August 12, 2025.
- Wilson, Steve, *The Developer's Playbook for Large Language Model Security: Building Secure AI Applications*, O'Reilly Media, 2024.
- Wu, Baoyuan, Mingli Zhu, Meixi Zheng, Zihao Zhu, Shaokui Wei, Mingda Zhang, Hongrui Chen, Danni Yuan, Li Liu, and Qingshan Liu, "Defenses in Adversarial Machine Learning: A Systematic Survey from the Lifecycle Perspective," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, Vol. 48, No. 1, January 2026.
- Xiao, Jiancong, Ziniu Li, Xingyu Xie, Emily Getzen, Cong Fang, Qi Long, and Weijie J. Su, "On the Algorithmic Bias of Aligning Large Language Models with RLHF: Preference Collapse and Matching Regularization," arXiv, arXiv:2405.16455v2, August 25, 2025.
- Xu, Hanhui, and Kyle Michael James Shuttleworth, "Medical Artificial Intelligence and the Black Box Problem: A View Based on the Ethical Principle of 'Do No Harm,'" *Intelligent Medicine*, Vol. 4, No. 1, February 2024.
- Yan, Biwei, Kun Li, Minghui Xu, Yueyan Dong, Yue Zhang, Zhaochun Ren, and Xiuzhen Cheng, "On Protecting the Data Privacy of Large Language Models (LLMs): A Survey," *2024 International Conference on Meta Computing (ICMC)*, June 2024.
- Yang, Jin, Zhiqiang Wang, Yanbin Lin, and Zunduo Zhao, "Problematic Tokens: Tokenizer Bias in Large Language Models," *2024 IEEE International Conference on Big Data (Big Data)*, December 2024.
- Yu, Guangba, Zirui Wang, Yujie Huang, Renyi Zhong, Yuedong Zhong, Yilun Wang, and Michael R. Lyu, "Why Does the LLM Stop Computing: An Empirical Study of User-Reported Failures in Open-Source LLMs," arXiv, arXiv:2601.13655v1, January 20, 2026.

- Zeng, Shenglai, Pengfei He, Kai Guo, Tianqi Zheng, Hanqing Lu, Yue Xing, and Hui Liu, "Towards Context-Robust LLMs: A Gated Representation Fine-Tuning Approach," *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, Vol. 1, July 2025.
- Zhao, Andrew, Reshmi Ghosh, Vitor Carvalho, Emily Lawton, Keegan Hines, Gao Huang, and Jack W. Stokes, "Are My Optimized Prompts Compromised? Exploring Vulnerabilities of LLM-Based Optimizers," arXiv, arXiv:2510.14381v2, last updated January 13, 2026.
- Zhou, Chunting, Pengfei Liu, Puxin Xu, Srini Iyer, Jiao Sun, Yuning Mao, Xuezhe Ma, Avia Efrat, Ping Yu, Lili Yu, et al., "LIMA: Less Is More for Alignment," *NIPS '23: Proceedings of the 37th International Conference on Neural Information Processing Systems*, December 2023.
- Zhu, Derui, Dingfan Chen, Xiongfei Wu, Jiahui Geng, Zhuo Li, Jens Grossklags, and Lei Ma, "PrivAuditor: Benchmarking Privacy Vulnerabilities in LLM Adaptation Techniques," *NIPS '24: Proceedings of the 38th International Conference on Neural Information Processing Systems*, December 2024.
- Zizzo, Giulio, Giandomenico Cornacchia, Kieran Fraser, Muhammad Zaid Hameed, Ambrish Rawat, Beat Buesser, Mark Purcell, Pin-Yu Chen, Prasanna Sattigeri, and Kush Varshney, "Adversarial Prompt Evaluation: Systematic Benchmarking of Guardrails Against Prompt Input Attacks on LLMs," arXiv, arXiv:2502.15427v1, February 21, 2025.
- Zverev, Egor, Sahar Abdelnabi, Soroush Tabesh, Mario Fritz, and Christoph H. Lampert, "Can LLMs Separate Instructions from Data? And What Do We Even Mean by That?" arXiv, arXiv:2403.06833v3, January 31, 2025.

About the Authors

Elie Alhajjar is a senior information scientist at RAND. Since joining RAND, he has been involved in multiple efforts around the commercial space sector, Army acquisition, adversarial machine learning, cyber risk, and countering weapons of mass destruction. He holds a Ph.D. in applied mathematics.

Sasha Romanosky is a senior policy researcher at RAND. His research areas include the economics of security, AI security, cyber insurance, cybercrime, and other national security matters. He holds a Ph.D. in public policy.

Kyle A. Kilian is a senior technical analyst at RAND. His research interests include emerging technological risks across AI security, cybersecurity, risk management, and defense and intelligence. He holds an M.A. in international affairs and an M.S. in cyber intelligence and data science.

Joe Uchill is an assistant policy researcher at RAND and a Ph.D. student in the RAND School of Public Policy. His primary research interests are cybersecurity policy, election security, critical infrastructure protection, cybercrime, and the cybercriminal economy. He holds an M.S. in journalism.